OpenMP

A collage of several tutorials

What is OpenMP?

- OpenMP = Open Multi-Parallelism
- It is an API to explicitly direct *multi-threaded shared-memory parallelism*.
- Comprised of three primary API components
  - Compiler directives
    - These define which parts of the code are run in parallel
  - Run-time library routines
    - Support routines such as identifying the thread ID
  - Environment variables
    - Used to adjust the runtime behaviour of the parallel application

The OpenMP* Common Core

Yun (Helen) He
LBL/NERSC

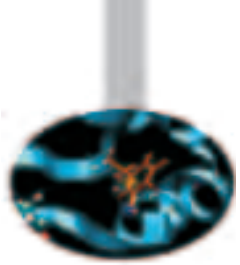
Ruud van der Pas
Oracle Linux Engineering

Tim Mattson
Intel

Alice Koniges
Maui HPC Center

The first version of the "Common Core" slides was created by Tim Mattson, Intel Corp. Many others (e.g. Barbara Chapman, Mark Bull, Larry Meadows, …) have contributed to these slides.

CINECA SCAI
SuperComputing Applications and Innovation



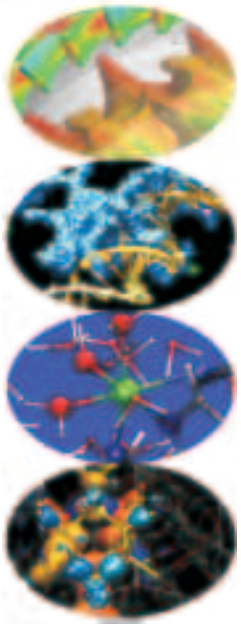
Introduction to OpenMP

Claudia Truini - c.truini@cineca.it

Vittorio Ruggiero - v.ruggiero@cineca.it

Mariella Ippolito - m.ippolito@cineca.it

SuperComputing Applications and Innovation Department



CINECA

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Computing

Prof. Marco Bertini

Outline

- **Introduction**
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

OpenMP is ideally suited for modern architectures

Portable, and portable, and portable

Memory and threading model map naturally

Tight integration with accelerator support

Lightweight

Mature, yet continues to evolve

Widespread support and still on the rise

OpenMP Overview

C$OMP FLUSH

#pragma omp critical

C$OMP THREADPRIVATE(/ABC/)

CALL OMP_SET_NUM_THREADS(10)

C$OMP PARALLEL COPYIN(/blk/)

C$OMP DO lastprivate(XX)

Nthrds = OMP_GET_NUM_PROCS()

omp_set_lock(lck)

OpenMP: An API for Writing Multithreaded Applications

- A set of compiler directives and library routines for parallel application programmers
- Greatly simplifies writing multi-threaded (MT) programs in Fortran, C and C++
- Standardizes established SMP practice + vectorization and heterogeneous device programming

* The name “OpenMP” is the property of the OpenMP Architecture Review Board.

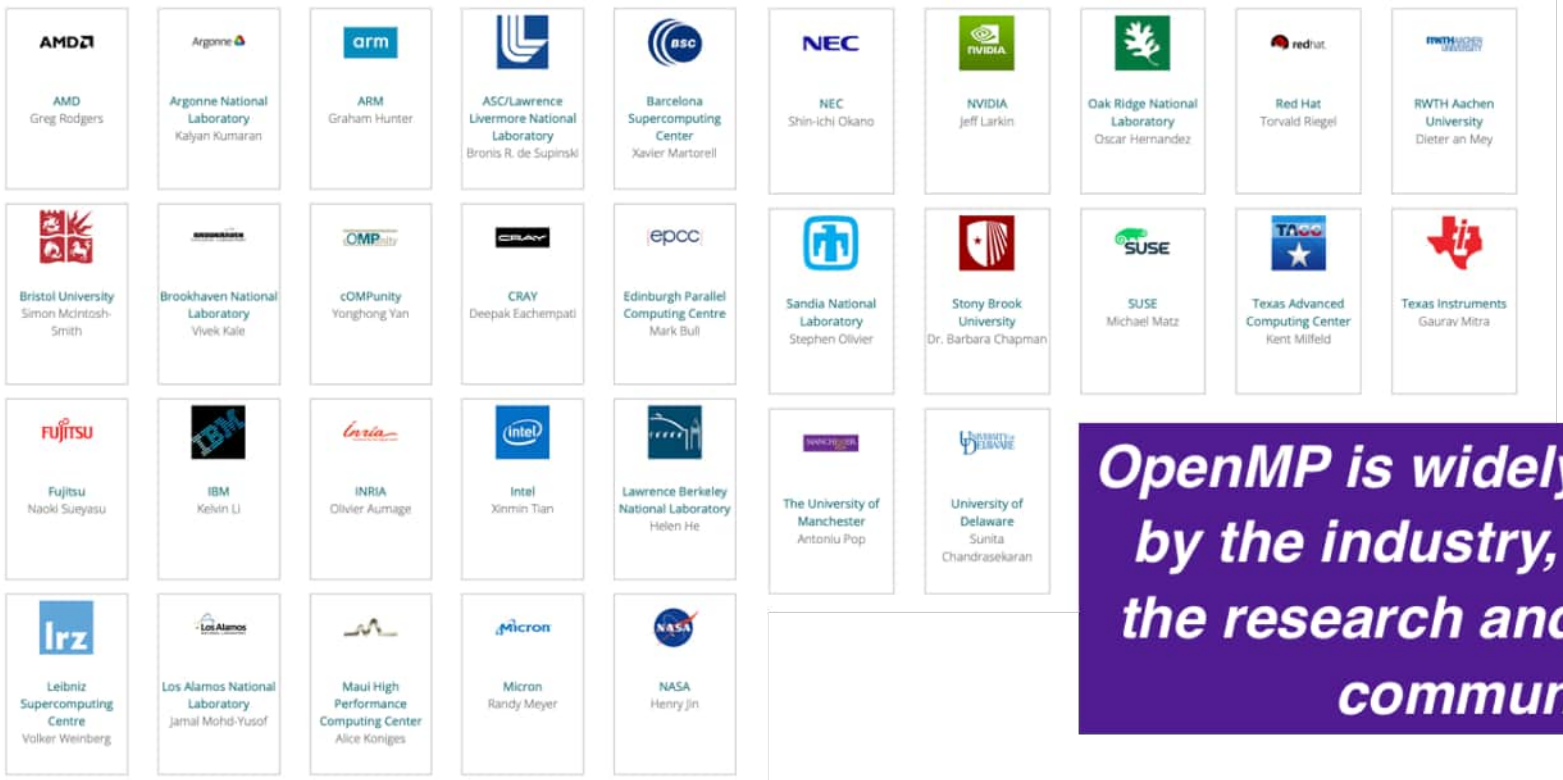
What OpenMP Really Is

- OpenMP is an industry standard API for C, C++, and Fortran to support shared memory, vectorization, and heterogeneous device programming
- Specifications are defined by the OpenMP Architecture Review Board (ARB)
  - 35+ members, including industry, government labs, and academia
  - Available in major commercial and open source compilers

The OpenMP ARB Mission

To standardize directive-based multi-language high-level parallelism that is performant, productive, and portable

Support Continues to Increase!

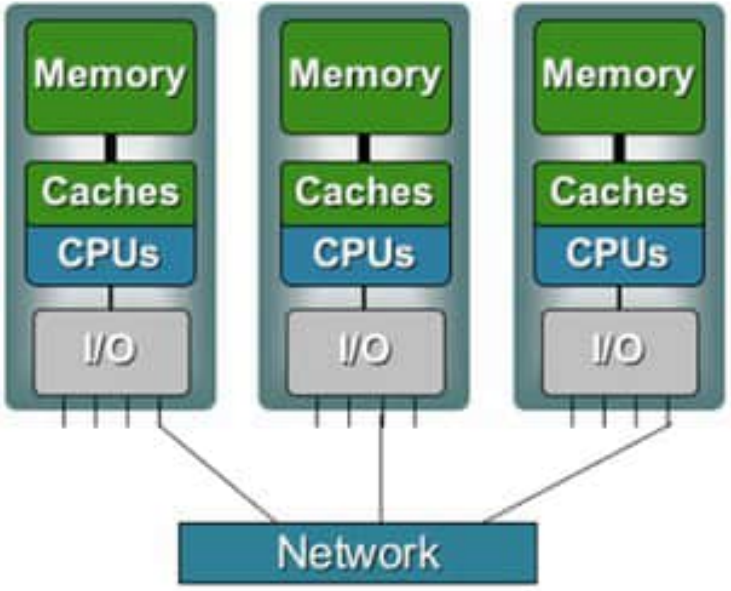


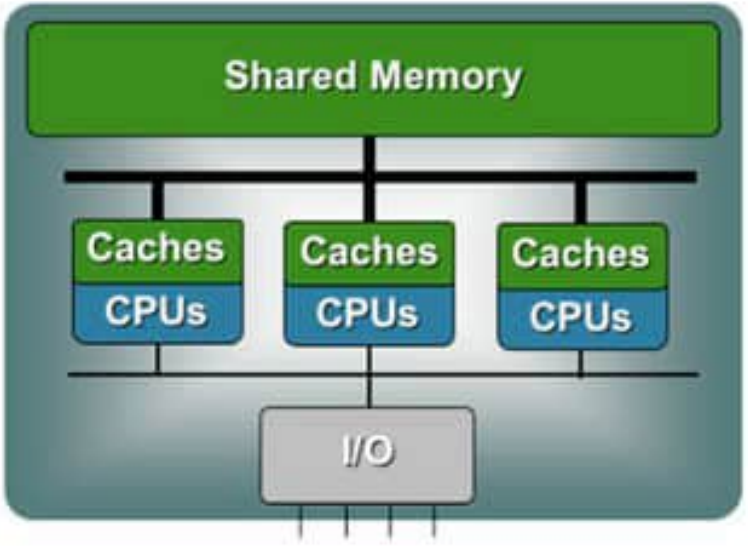
OpenMP is widely supported by the industry, as well as the research and academic community

http://www.openmp.org

Shared memory architectures

- All processors may access the whole main memory

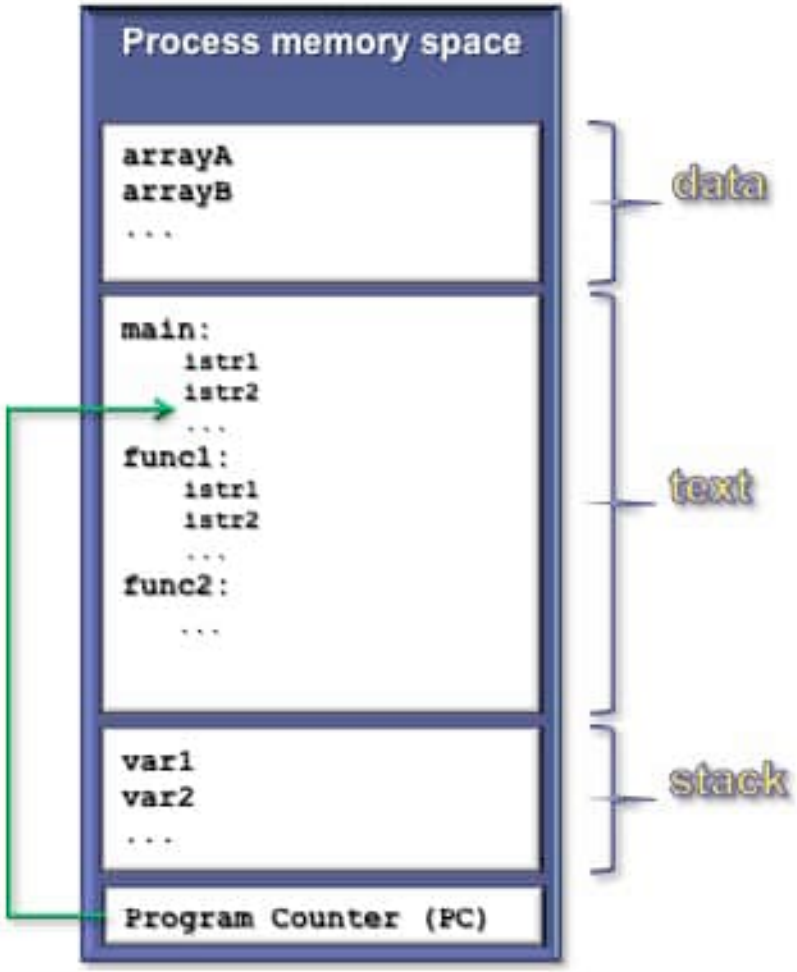




- Non-Uniform Memory Access
  - Memory access time is non-uniform
- Uniform Memory Access
  - Memory access time is uniform

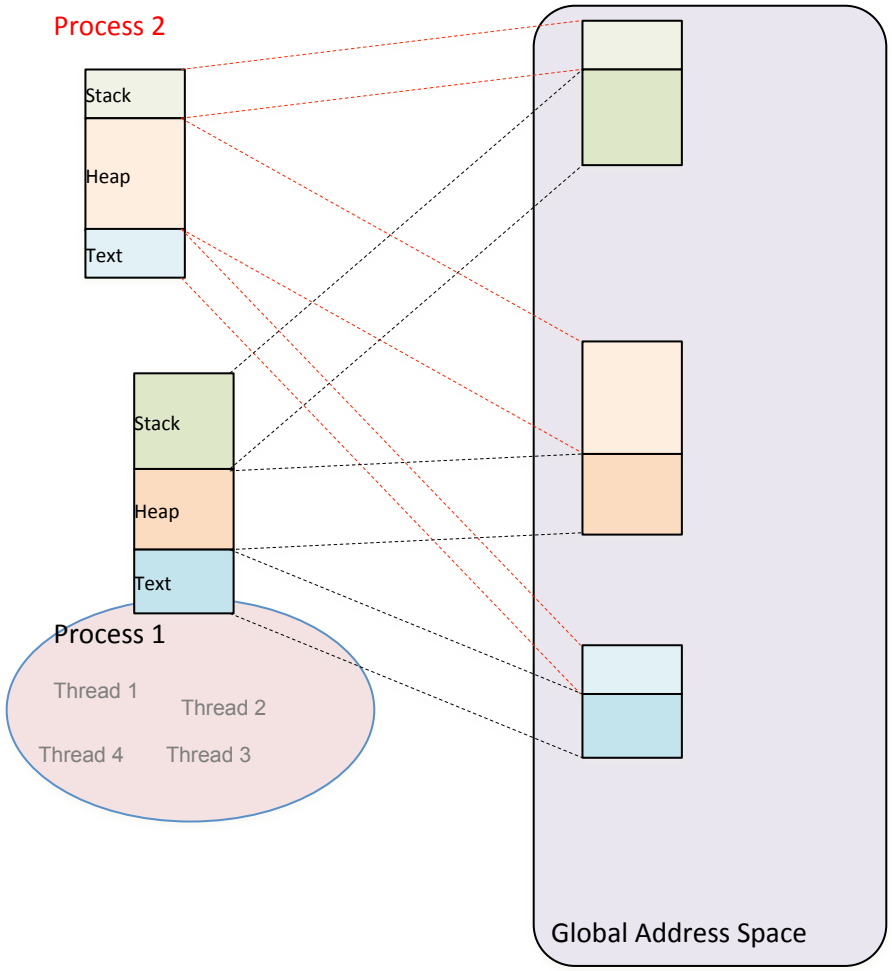
Process and thread

- A process is an instance of a computer program
- Some information included in a process are:
  - Text
    - Machine code
  - Data
    - Global variables
  - Stack
    - Local variables
  - Program counter (PC)
    - A pointer to the istruction to be executed



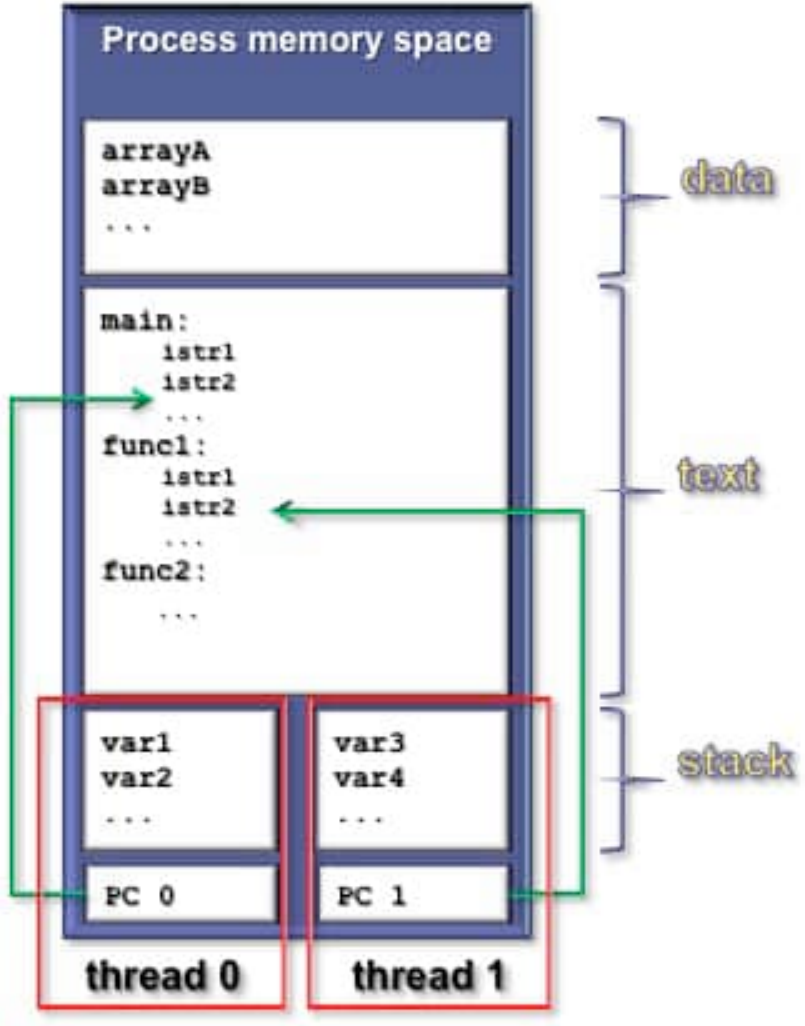
Operating System Memory Model

- A process, owns a lot of state information including the memory, file handles, etc.
- Operating systems provide a separate address space for each process
- One process cannot see the memory of another process
- Memory within a process is managed separately between
  - text and data segment - static read only areas for code and data known at compile time
  - the heap - an area of memory managed by the operating system for dynamic data allocation
  - the stack - a piece of memory managed by the process itself
- Multiple threads launched from the same process share the same address space
- Threads, launched by the process, share the state information, including memory, of the launching process and so are considered *light weight tasks*



Multi-threading

- The process contains several concurrent execution flows (threads)
  - Each thread has its own program counter (PC)
  - Each thread has its own private stack (variables local to the thread)
  - The instructions executed by a thread can access:
    - the process global memory (data)
    - the thread local stack



Why use Threading?

- Exploit *more* parallelism to increase scaling and performance
  - SMPD parallelism
    - E.g. Distributed memory (e.g. MPI) tasks on one spatial dimension, threads (e.g. OpenMP) on another.
    - Or, Reduce number of distributed mempory tasks thus reducing communications overhead.
  - Functional parallelism
    - Threads executing different tasks, perhaps on the same data, perhaps not.
  - Improve load balance via constructs that enable *work stealing*.
  - Reduce memory overheads from many separate processes by using lightweight threads which share memory

Alternatives 1 - Pthreads

- An alternative model for combining threads with message passing is to use the POSIX Threads interface and library Pthreads
- Most OpenMP implementations are built on top of Pthreads !!
  - … in fact most threading libraries of any type are built on Pthreads
- Pthreads provides the ability to dynamically create threads which are launched to run a specific task
- Pthreads provides finer grained control than OpenMP
  - Uses ***mutex*** and ***condition*** variables for synchronisation
- The disadvantages of Pthreads compared to OpenMP are
  - There is no Fortran interface in the standard
    - Although IBM did produce a Fortran interface for their XLF compiler
  - You have to manage (re-create) any worksharing yourself
  - It is a low level interface
    - As MPI is often referred to as a low level interface this might not be a problem
  - You will typically take many more lines of coding using Pthreads than OpenMP
    - OpenMP is normally a more *productive* way of coding for numerical codes

Alternatives 2 - TBB

- Intel Thread Building Blocks (TBB) is a C++ template library that adds parallel programming for C++ programmers.
  - Not applicable to Fortran or C codes
- "Extends" C++ by adding a set of parallel keywords
  - These "extensions" are not actually real changes to the languages
  - As a template library, any standard-conforming C++ compiler can use TBB
- Algorithms can be parallelised in a number of ways
  - `parallel_for`, `parallel_reduce`, `parallel_while` etc.
- TBB adds containers, locks, a task scheduler etc.
- TBB was built out of positive user experiences from OpenMP, and the desire to provide an object-oriented, template based approach to parallelism in C++

TBB vs OpenMP

- Note – Intel not only provides TBB but it also has many members, directors and officers of the OpenMP forum
- Here are some suggestions or tips provided on the TBB website
  - "Everyone should use OpenMP as much as they can. It is easy to use, it is standard, it is supported by all major compilers, and it exploits parallelism well."
  - "OpenMP *[is]* the standard way to do parallelism in C and Fortran."
  - "Use OpenMP if the parallelism is primarily for bounded loops over built-in types, or if it is flat do-loop centric parallelism. … It can be very challenging to match OpenMP performance with TBB for such problems. It is seldom worth the effort to bother – just use OpenMP."
  - "Should I expect TBB to outperform OpenMP and MPI?
    No, TBB may offer a competitive alternative but in general TBB exists to help where OpenMP cannot, and to be far easier to program than MPI."
  - "OpenMP and MPI continue to be good choices in High Performance Computing applications; TBB has been designed to be more conducive to application parallelization on client platforms such as laptops and desktops, going beyond data parallelism …"
- TBB might be a good choice for C++ programmers if their code does not fit the standard data-parallel model
- Note also that TBB and OpenMP can coexist

So Why OpenMP ?

- OpenMP is ubiquitous
  - Available with almost all compilers
    - gcc provides OpenMP support
      - Therefore anyone can get hold of an OpenMP enabled C/Fortran compiler for free
      - GCC 4.7 provides full support for OpenMP 3.1 standard
    - Vendor tuned implementations available
- Easy interface available for incremental parallelism
- Is the best fit for data-parallel codes
- Is being updated to incorporate methods for modern programming methods and technologies
  - Task parallel features were added in OpenMP 3.0
  - Current roadmap suggests that accelerator directives will be added in OpenMP 4.0

Outline

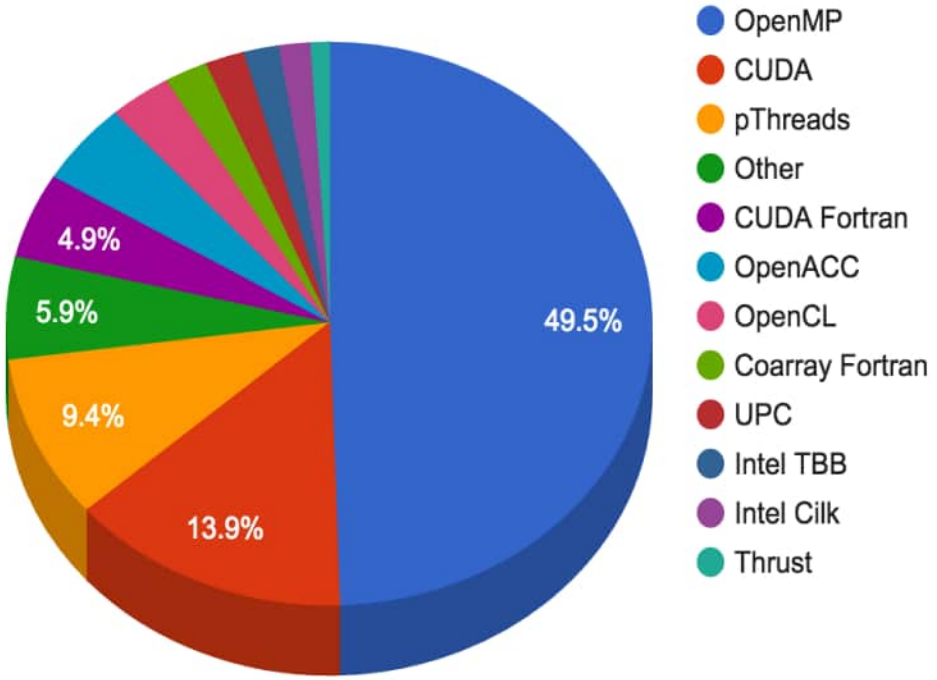
- Introduction
- **Why Common Core?**
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

Choice of Programming Models for Modern HPC Systems

- MPI was developed primarily for inter-address space (inter means between or among)
- OpenMP was developed for **shared memory** or intra-node (intra means within)
  - Trend: multi-socket nodes with rapidly increasing core counts
  - Trend: accelerators
- Hybrid Programming (**MPI+X**) is when we use a solution with different programming models for inter *vs.* intra-node parallelism

What is X in "MPI+X"?
OpenMP is about 50% in 2015 at NERSC

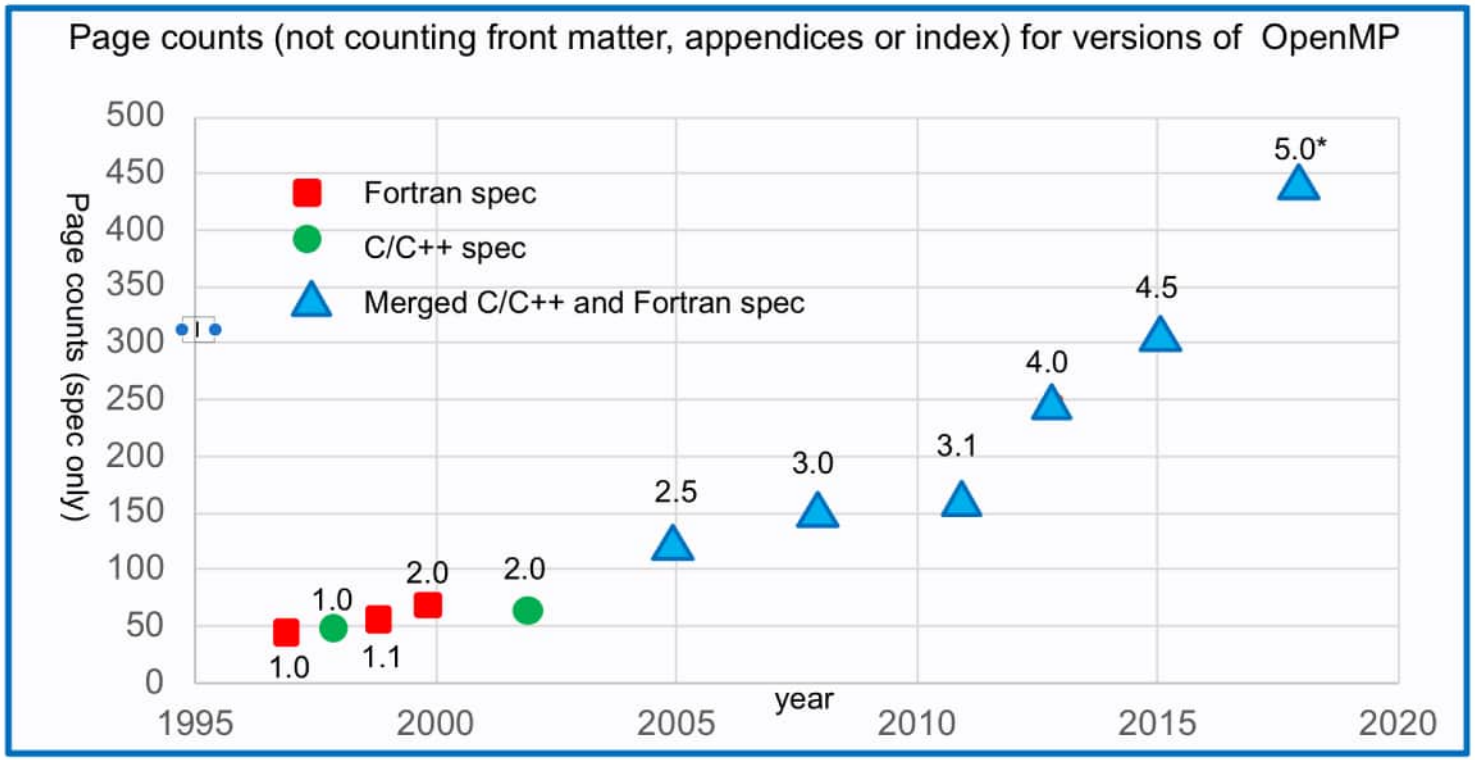
Update in 2017: OpenMP increases to 75%



The National Energy Research Scientific Computing Center (NERSC) is the primary scientific computing facility for the Office of Science in the U.S. Department of Energy.

Why the Common Core?

- OpenMP started out in 1997 as a simple interface for the application programmers more versed in their area of science than computer science.
- The complexity has grown considerably over the years!



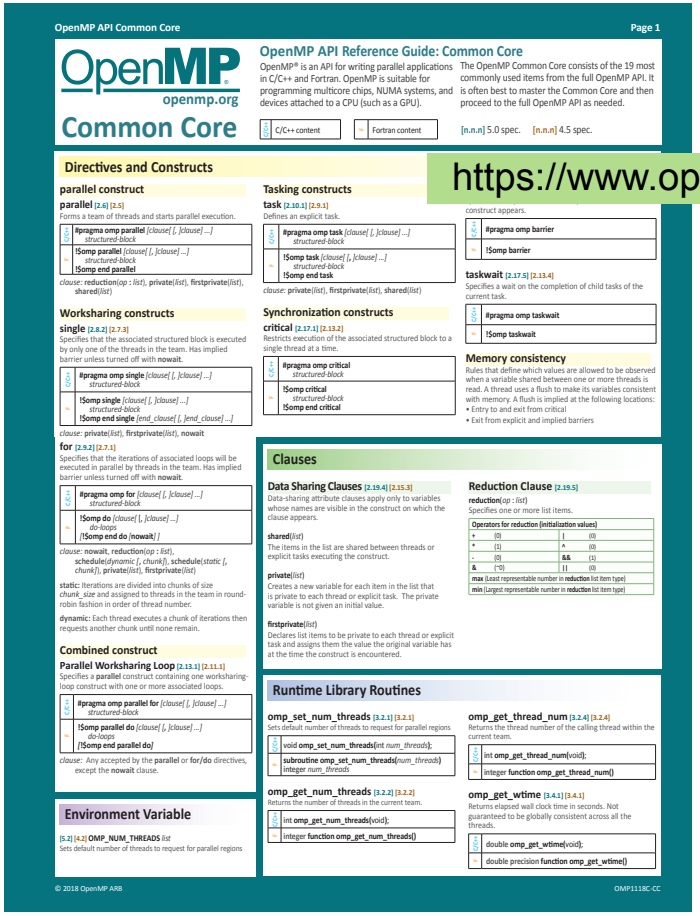
* Does not include the tools interface added with OpneMP 5.0 which pushes the page count to 618

The complexity of the full spec is overwhelming, so we focus on the 21 items most OpenMP programmers restrict themselves to, the so called “OpenMP Common Core”

The OpenMP Common Core: Most OpenMP Programs Only Use These 21 Items

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads.
void omp_set_thread_num() int omp_get_thread_num() int omp_get_num_threads()	Default number of threads and internal control variables. SPMD pattern: Create threads with a parallel region and split up the work using the number of threads and the thread ID.
double omp_get_wtime()	Speedup and Amdahl's law. False sharing and other performance issues.
setenv OMP_NUM_THREADS N	Setting the internal control variable for the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies.
reduction(op:list)	Reductions of values across a team of threads.
schedule (static [,chunk]) schedule(dynamic [,chunk])	Loop schedules, loop overheads, and load balance.
shared(list), private(list), firstprivate(list)	Data environment.
default(none)	Force explicit definition of each variable's storage attribute
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive).
#pragma omp single	Workshare with a single thread.
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.

Common Core Reference Guide



OpenMP API Common Core — Page 1

OpenMP API Reference Guide: Common Core

openmp.org

Common Core

OpenMP® is an API for writing parallel applications in C/C++ and Fortran. OpenMP is suitable for programming multicore chips, NUMA systems, and devices attached to a CPU (such as a GPU).

The OpenMP Common Core consists of the 19 most commonly used items from the full OpenMP API. It is often best to master the Common Core and then proceed to the full OpenMP API as needed.

C/C++ content | Fortran content

[n.n.n] 5.0 spec. [n.n.n] 4.5 spec.

https://www.openmp.org/specifications/

Directives and Constructs

parallel construct

parallel [2.6] [2.5]

Forms a team of threads and starts parallel execution.

C/C++: #pragma omp parallel [clause[[,]clause] ...] structured-block

Fortran: !$omp parallel [clause[[,]clause] ...] structured-block !$omp end parallel

clause: reduction(op : list), private(list), firstprivate(list), shared(list)

Worksharing constructs

single [2.8.2] [2.7.3]

Specifies that the associated structured block is executed by only one of the threads in the team. Has implied barrier unless turned off with nowait.

C/C++: #pragma omp single [clause[[,]clause] ...] structured-block

Fortran: !$omp single [clause[[,]clause] ...] structured-block !$omp end single [end_clause[[,]end_clause] ...]

clause: private(list), firstprivate(list), nowait

for [2.9.2] [2.7.1]

Specifies that the iterations of associated loops will be executed in parallel by threads in the team. Has implied barrier unless turned off with nowait.

C/C++: #pragma omp for [clause[[,]clause] ...] structured-block

Fortran: !$omp do [clause[[,]clause] ...] do-loops [!$omp end do [nowait]]

clause: nowait, reduction(op : list), schedule(dynamic [, chunk]), schedule(static [, chunk]), private(list), firstprivate(list)

static: Iterations are divided into chunks of size chunk_size and assigned to threads in the team in round-robin fashion in order of thread number.

dynamic: Each thread executes a chunk of iterations then requests another chunk until none remain.

Combined construct

Parallel Worksharing Loop [2.13.1] [2.11.1]

Specifies a parallel construct containing one worksharing-loop construct with one or more associated loops.

C/C++: #pragma omp parallel for [clause[[,]clause] ...] structured-block

Fortran: !$omp parallel do [clause[[,]clause] ...] do-loops [!$omp end parallel do]

clause: Any accepted by the parallel or for/do directives, except the nowait clause.

Environment Variable

[5.2] [4.2] OMP_NUM_THREADS list

Sets default number of threads to request for parallel regions

Tasking constructs

task [2.10.1] [2.9.1]

Defines an explicit task.

C/C++: #pragma omp task [clause[[,]clause] ...] structured-block

Fortran: !$omp task [clause[[,]clause] ...] structured-block !$omp end task

clause: private(list), firstprivate(list), shared(list)

Synchronization constructs

critical [2.17.1] [2.13.2]

Restricts execution of the associated structured block to a single thread at a time.

C/C++: #pragma omp critical structured-block

Fortran: !$omp critical structured-block !$omp end critical

construct appears.

C/C++: #pragma omp barrier

Fortran: !$omp barrier

taskwait [2.17.5] [2.13.4]

Specifies a wait on the completion of child tasks of the current task.

C/C++: #pragma omp taskwait

Fortran: !$omp taskwait

Memory consistency

Rules that define which values are allowed to be observed when a variable shared between one or more threads is read. A thread uses a flush to make its variables consistent with memory. A flush is implied at the following locations:

- Entry to and exit from critical
- Exit from explicit and implied barriers

Clauses

Data Sharing Clauses [2.19.4] [2.15.3]

Data-sharing attribute clauses apply only to variables whose names are visible in the construct on which the clause appears.

shared(list)

The items in the list are shared between threads or explicit tasks executing the construct.

private(list)

Creates a new variable for each item in the list that is private to each thread or explicit task. The private variable is not given an initial value.

firstprivate(list)

Declares list items to be private to each thread or explicit task and assigns them the value the original variable has at the time the construct is encountered.

Reduction Clause [2.19.5]

reduction(op : list)

Specifies one or more list items.

Operators for reduction (initialization values)			
+	(0)	|	(0)
*	(1)	^	(0)
-	(0)	&&	(1)
&	(~0)	||	(0)

max (Least representable number in reduction list item type)

min (Largest representable number in reduction list item type)

Runtime Library Routines

omp_set_num_threads [3.2.1] [3.2.1]

Sets default number of threads to request for parallel regions

C/C++: void omp_set_num_threads(int num_threads);

Fortran: subroutine omp_set_num_threads(num_threads) integer num_threads

omp_get_num_threads [3.2.2] [3.2.2]

Returns the number of threads in the current team.

C/C++: int omp_get_num_threads(void);

Fortran: integer function omp_get_num_threads()

omp_get_thread_num [3.2.4] [3.2.4]

Returns the thread number of the calling thread within the current team.

C/C++: int omp_get_thread_num(void);

Fortran: integer function omp_get_thread_num()

omp_get_wtime [3.4.1] [3.4.1]

Returns elapsed wall clock time in seconds. Not guaranteed to be globally consistent across all the threads.

C/C++: double omp_get_wtime(void);

Fortran: double precision function omp_get_wtime()

OMP1118C-CC

OpenMP On Your Laptop (1)

- Mac OS X/Sierra
  - Compiler installed in native XCode doesn't "do" OpenMP
  - Install "homebrew"
    - https://brew.sh/
  - Install GNU Compiler Collection
    - $ brew update
    - $ brew install gcc **(this will take a long time)**
  - Use the compilers! **(note the command names)**
    - $ gcc-9 -fopenmp program.c
    - $ g++-9 -fopenmp program.cpp
    - $ gfortran-9 -fopenmp program.f90
- Windows
  - Install free version of Visual Studio
    - https://www.visualstudio.com/downloads/

OpenMP On Your Laptop (2)

- Linux
  - Install GNU Compiler Collection (C, C++, Fortran)
    - Ubuntu-like: C, C++ already there by default
      - # apt install gfortran
    - CentOS-like
      - # yum install gcc-c++ gcc-gfortran
    - Fedora-like
      - # dnf install gcc-c++ gcc-gfortran
  - Use compilers!
    - $ gcc -fopenmp program.c
    - $ g++ -fopenmp program.cpp
    - $ gfortran -fopenmp program.f90

Outline

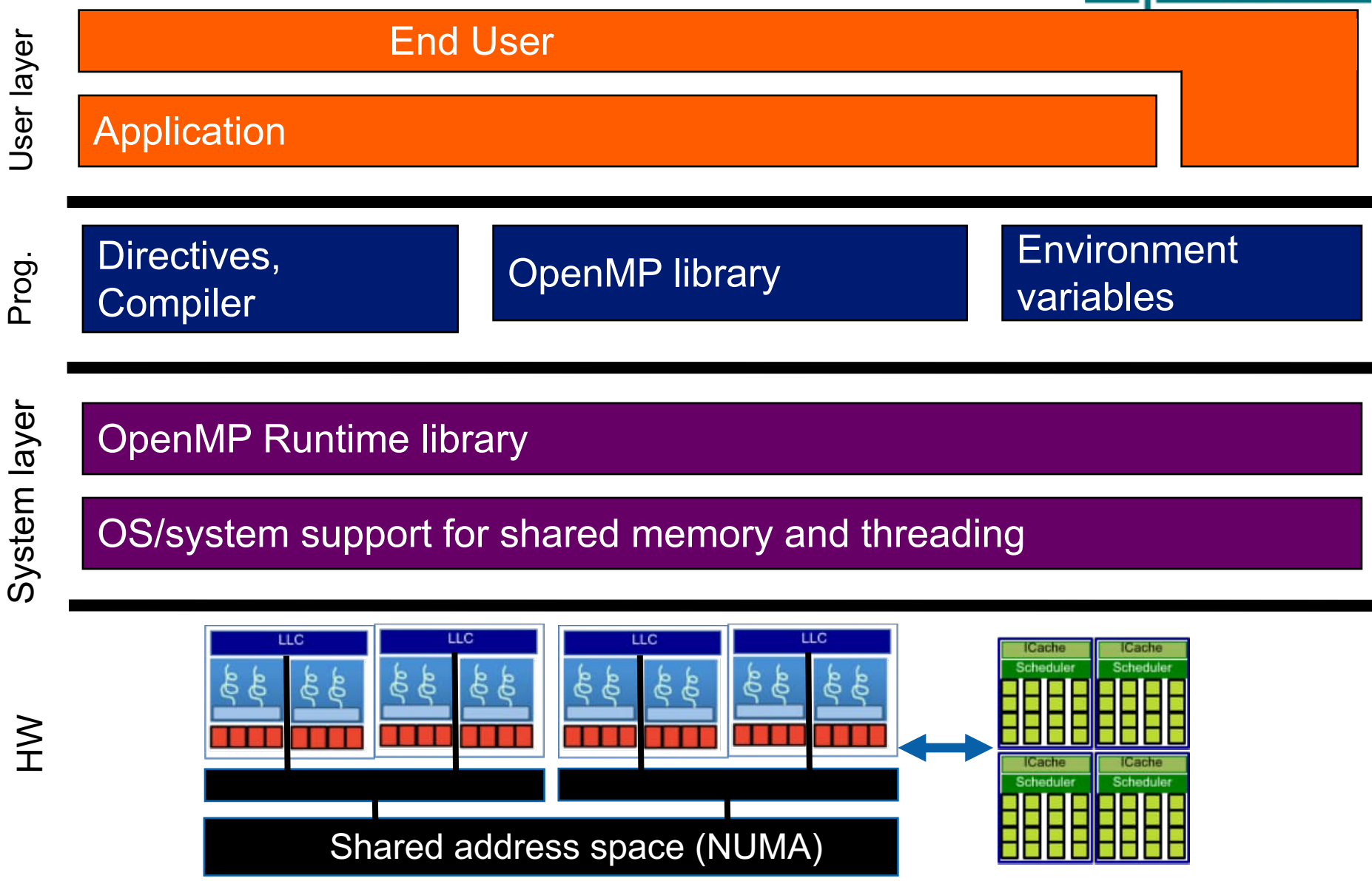
- Introduction
- Why Common Core?
- **OpenMP Basics**
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

How Does OpenMP Work?

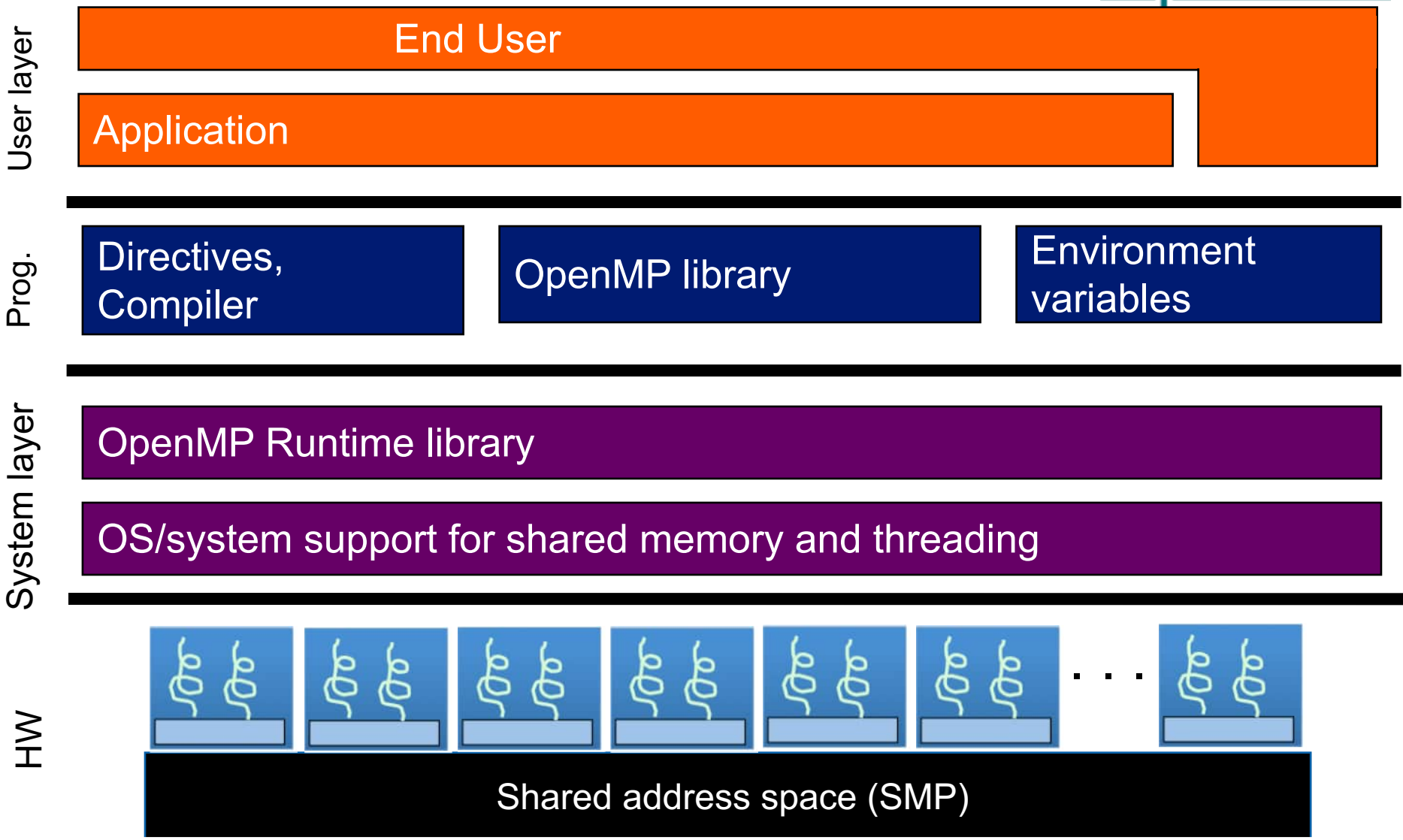
- Teams of OpenMP threads are created to perform the computation in a code
  - **Work is divided** among the threads, which run on the different cores
  - The threads collaborate **by sharing variables**
  - Threads **synchronize** to order accesses and prevent data corruption
  - **Structured programming** is encouraged to reduce likelihood of bugs

- OpenMP components:
  - Compiler Directives and Clauses
  - Runtime Libraries
  - Environment Variables

OpenMP Basic Definitions: Basic Solution Stack



OpenMP Basic Definitions: Basic Solution Stack



Fort the OpenMP Common Core, we focus on Symmetric Multiprocessor Case ….
i.e. lots of threads with “equal cost access” to memory

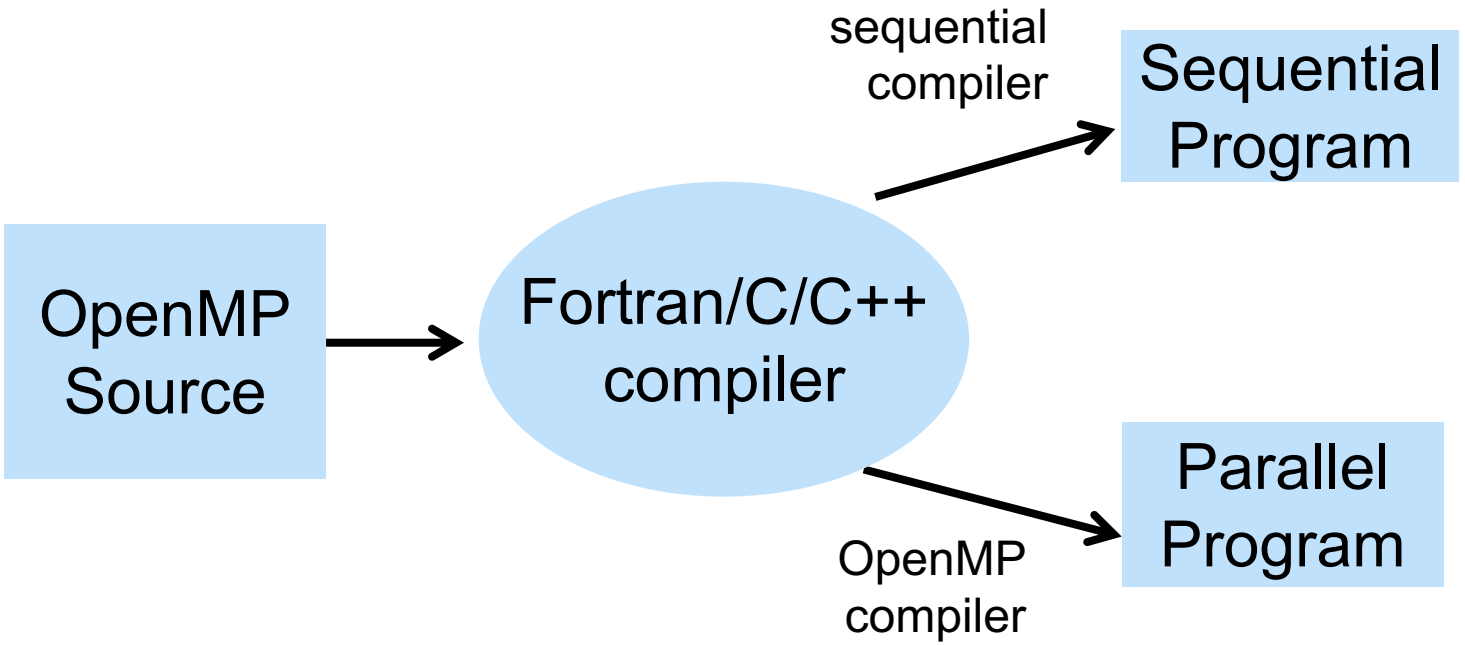
OpenMP Basic Syntax

- Most of the constructs in OpenMP are compiler directives.

C and C++	Fortran
Compiler directives	
#pragma omp construct [clause [clause]…]	***!$OMP construct [clause [clause] …]***
Example	
#pragma omp parallel private(x) ***{*** ***}***	***!$OMP PARALLEL*** ***!$OMP END PARALLEL***
Function prototypes and types:	
#include <omp.h>	***use OMP_LIB***

- Most OpenMP constructs apply to a “structured block”.
  - Structured block: a block of one or more statements with one point of entry at the top and one point of exit at the bottom.
  - It’s OK to have an exit() within the structured block.

OpenMP Usage

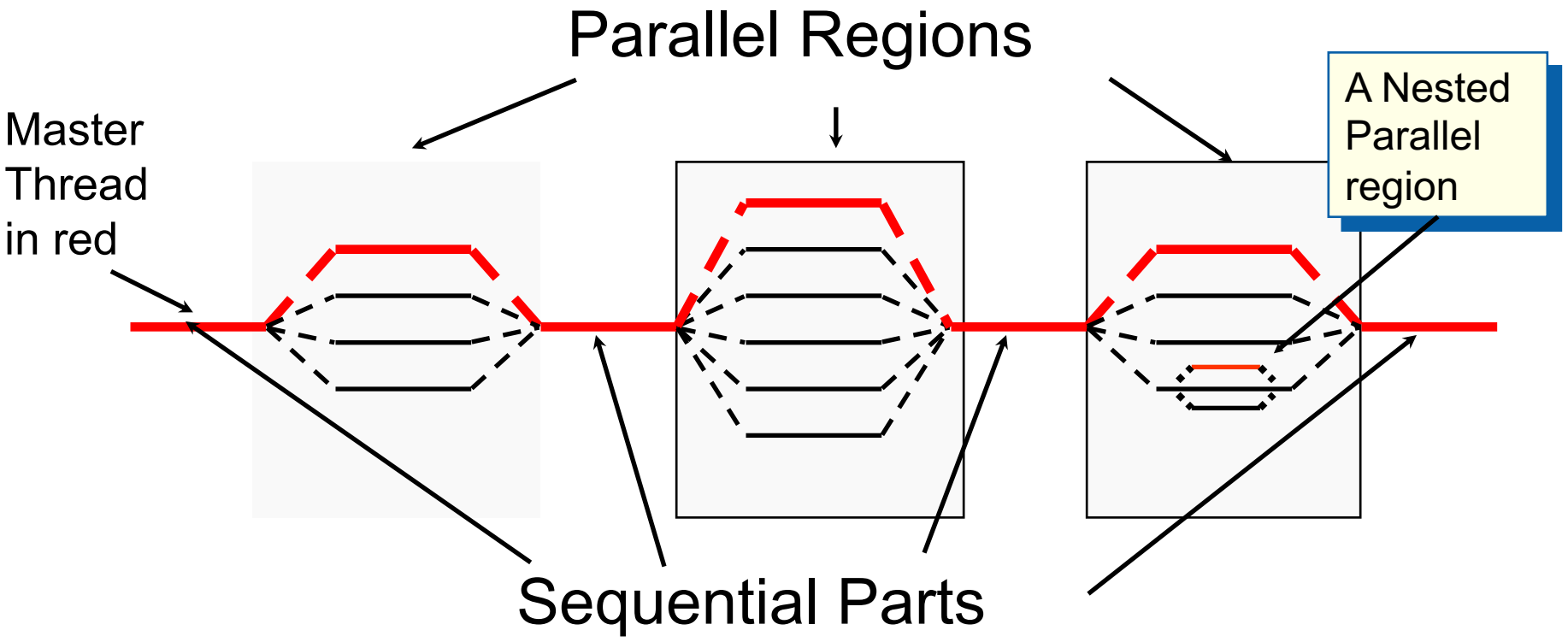


- Maintains single source code for serial and parallel programs.
- Needs special flag to enable OpenMP, such as:
  - Intel compiler: -qopenmp
  - GNU compiler: -fopenmp
  - PGI compiler: -mp
  - Cray compiler: none

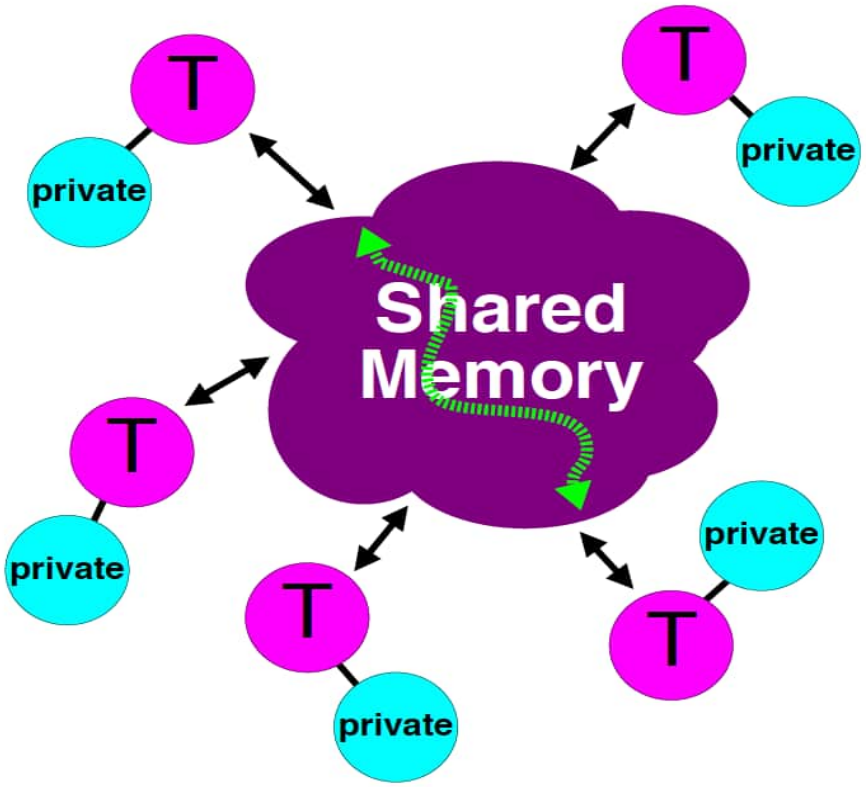
OpenMP Programming Model

Fork-Join Parallelism:

- Master thread spawns a team of threads as needed.
- Parallelism added incrementally until performance goals are met, i.e., the sequential program evolves into a parallel program.



OpenMP Memory Model



- ✔ *All threads have access to the same, globally shared, memory*
- ✔ *Data can be shared or private*
- ✔ *Shared data is accessible by all threads*
- ✔ *Private data can only be accessed by the thread that owns it*
- ✔ *Data transfer is transparent to the programmer*
- ✔ *Synchronization takes place, but it is mostly implicit*

Serial *vs.* OpenMP

Serial

```
void main ()
{
   double x[256];
   for (int i=0; i<256; i++)
       {
       some_work(x[i]);
       }
}
```

OpenMP

```
#include "omp.h"
void main ()
{
   double x(256);
#pragma omp parallel for
   for (int i=0; i<256; i++)
       {
       some_work(x(i));
       }
}
```

OpenMP is not just about parallelizing loops!
It offers a lot more ….

Advantages of OpenMP

- Simple programming model
  - Data decomposition and communication handled by compiler directives
- Single source code for serial and parallel codes
  - No major redesign of the serial code
- Portable implementation
- Easy to get started: incremental parallelization
  - Start from most critical or time consuming part of the code

A Multi-Threaded "Hello World" Program

- Write a multithreaded program where each thread prints "hello world".

```
#include <omp.h>
#include <stdio.h>
int  main()
{

#pragma omp parallel
 {

    printf(" hello ");
    printf(" world \n");
  }
}
```

OpenMP include file

Parallel region with default number of threads

End of the Parallel region

Sample Output:

hello hello world

world

hello hello world

world

The statements are interleaved based on how the operating schedules the threads

A Simple OpenMP Program

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main () {
    int tid, nthreads;
#pragma omp parallel private(tid)
   {
    tid = omp_get_thread_num();
    printf("Hello World from thread %d\n", tid);
#pragma omp barrier
     if ( tid == 0 ) {
        nthreads = omp_get_num_threads();
        printf("Total threads= %d\n",nthreads);
     }
  }
}
```

Sample Compile and Run:

```
% gcc -fopenmp test.c
% export OMP_NUM_THREADS=4
% ./a.out
```

```
Program main
use omp_lib       (or: include "omp_lib.h")
integer :: id, nthreads
!$OMP PARALLEL PRIVATE(id)
    id = omp_get_thread_num()
    write (*,*) "Hello World from thread", id
!$OMP BARRIER
    if ( id == 0 ) then
        nthreads = omp_get_num_threads()
        write (*,*) "Total threads=",nthreads
     end if
!$OMP END PARALLEL
End program
```

The statements are interleaved based on how the operating schedules the threads

Sample Output: (no specific order)

```
Hello World from thread        0
Hello World from thread        2
Hello World from thread        3
Hello World from thread        1
Total threads=       4
```

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- **Creating Threads**
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

Thread Creation: the parallel Construct

FORTRAN:

```
!$OMP PARALLEL PRIVATE(id)
   id = omp_get_thread_num()
   write (*,*) "I am thread", id
!$OMP END PARALLEL
```

C/C++:

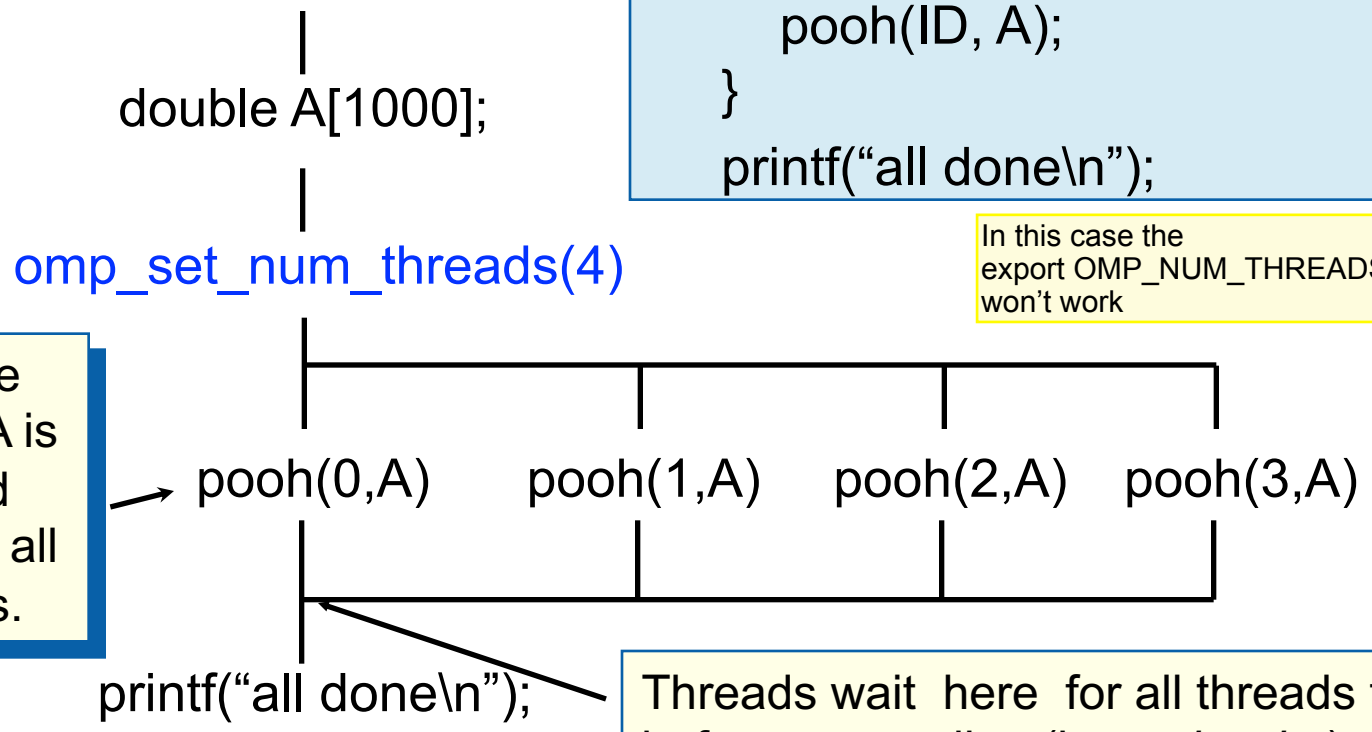
```
#pragma omp parallel private(thid)
{
    thid = omp_get_thread_num();
    printf("I am thread %d\n",
    thid);
}
```

- **A team of threads for parallel execution can only be created with the parallel construct.**
- **Each thread executes codes within the OpenMP parallel region.**
- **Threads wait at the end of parallel construct until all threads are finished with the parallel region before any proceed past the end of the parallel region.**

Parallel Regions Example

- Each thread executes the same code redundantly.

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    pooh(ID, A);
}
printf("all done\n");
```



In this case the export OMP_NUM_THREADS=xxx won't work

A single copy of A is shared between all threads.

Threads wait here for all threads to finish before proceeding (i.e., a *barrier*)

Single Worksharing Construct

- The **single** construct denotes a block of code that is executed by only one thread (not necessarily the master thread).
- A barrier is implied at the end of the single block (can remove the barrier with a *nowait* clause).

```
#pragma omp parallel
{
      do_many_things();
       #pragma omp single
        {exchange_boundaries();}
      do_many_other_things();
}
```

Various Methods to Set #threads

1) Use num_threads clause

```
#pragma omp parallel num_threads (4)
{
    int ID = omp_get_thread_num();
    pooh(ID,A);
}
```

2) Call omp_set_num_threads API

```
omp_set_num_threads(4);
#pragma omp parallel
{
   int ID = omp_get_thread_num();
   pooh(ID,A);
}
```

3) Set runtime environment

```
export OMP_NUM_THREADS=4
#pragma omp parallel
{
   int ID = omp_get_thread_num();
   pooh(ID,A);
}
```

4) Do none of the three above.

Code will use an implementation dependent default number of threads defined by the compiler.

- **Precedence: 1) > 2) > 3) > 4)**
- **You may get fewer threads than you requested, check with omp_get_num_threads()**

Thread creation: How Many Threads Did You Actually Get?

- You create a team threads in OpenMP with the parallel construct.
- You can request a number of threads with omp_set_num_threads()
- But is the number of threads requested the number you actually get?
  - NO! An implementation can silently decide to give you a team with fewer threads.
  - Once a team of threads is established … the system will not reduce the size of the team.

Each thread executes a copy of the code within the structured block

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
        int ID      = omp_get_thread_num();
        int nthrds = omp_get_num_threads();
        pooh(ID,A);
}
```

Runtime function to request a certain number of threads

Runtime function to return actual number of threads in the team

- Each thread calls pooh(ID,A) for `ID` = `0` to `nthrds-1`

Performance Tips

- Experiment to find the best number of threads on your system
- Put as much code as possible inside parallel regions
  - Amdahl's law: **If 1/s of the program is sequential, then you cannot ever get a speedup better than s**
  - So if 1% of a program is serial, speedup is limited to 100, no matter how many processors it is computed on
- Have large parallel regions
  - Minimize overheads: starting and stopping threads, executing barriers, moving data into cache
  - Directives can be "orphaned"; procedure calls inside regions are fine
- Run-time routines are your friend
  - Usually very efficient and allow maximum control over thread behavior
- Barriers are expensive
  - With large numbers of threads, they can be slow
  - Depends in part on HW and on implementation quality
  - Some threads might have to wait a long time if load not balanced

Orphaned Directives

Orphaning is a situation when directives related to a parallel region are not required to occur lexically within a single program unit.

Directives such as CRITICAL, BARRIER, SECTIONS, SINGLE, MASTER, and DO (FOR in C) can occur by themselves in a program unit, dynamically "binding" to the enclosing parallel region at run time.

Orphaned directives enable parallelism to be inserted into existing code with a minimum of code restructuring. Orphaning can also improve performance by enabling a single parallel region to bind with multiple DO directives located within called subroutines.

```
...
!$OMP PARALLEL
CALL PHASE1
CALL PHASE2
!$OMP END PARALLEL
...

SUBROUTINE PHASE1
!$OMP DO PRIVATE(i) SHARED(n)
DO i = 1, n
 CALL SOME_WORK(i)
END DO
!$OMP END DO
END

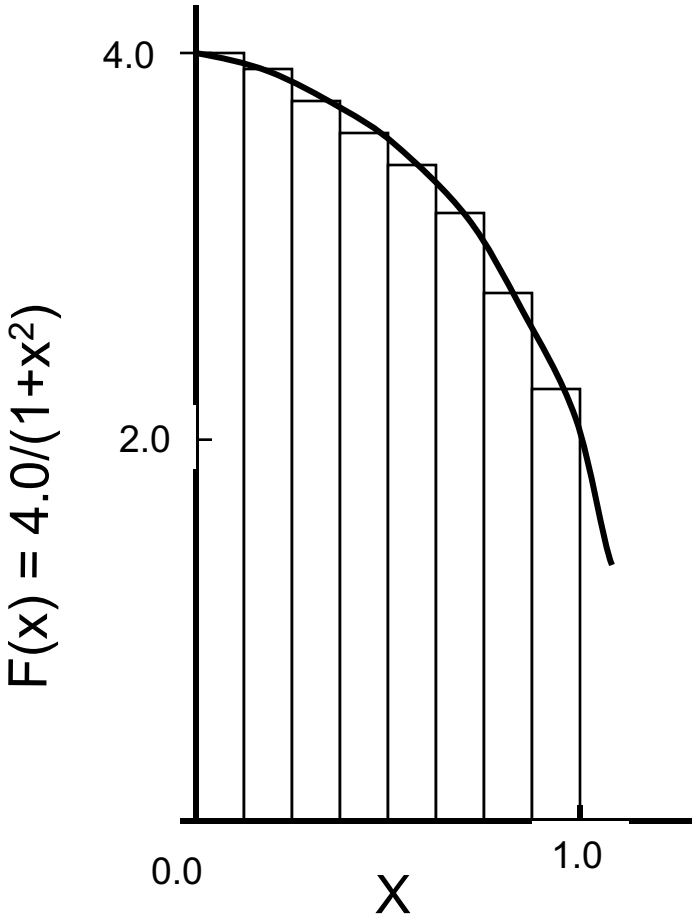
SUBROUTINE PHASE2
!$OMP DO PRIVATE(j) SHARED(n)
DO j = 1, n
 CALL MORE_WORK(j)
END DO
!$OMP END DO
END
```

Run-time routines

`void omp_set_num_threads(int num_threads)`	Dynamically set the number of threads to use for this region.
`int omp_get_num_threads(void)`	Determine what the current number of threads is that is allowed to execute a region.
`int omp_get_max_threads(void)`	Obtains the maximum number of threads ever allowed with this OpenMP* implementation.
`int omp_get_thread_num(void)`	Determines the unique thread number of the thread currently executing this section of code.
`int omp_get_num_procs(void)`	Determines the number of processors on the current machine.
`int omp_in_parallel(void)`	Returns non-zero if it is called within the dynamic extent of a parallel region executing in parallel, otherwise it returns zero.
`void omp_set_dynamic(int dynamic_threads)`	Enable or disable dynamic adjustment of the number of threads used to execute a parallel region. If `dynamic_threads` is non-zero, dynamic threads are enabled. If `dynamic_threads` is zero, dynamic threads are disabled.
`int omp_get_dynamic(void)`	Returns non-zero if dynamic thread adjustment is enabled and returns zero otherwise.
`void omp_set_nested(int nested)`	Enable or disable nested parallelism. If parameter is non-zero, enable. Default is disabled.
`int omp_get_nested(void)`	Always returns zero in the current version of compiler.
`void omp_init_lock(omp_lock_t *lock)`	Initialize a unique lock and set lock to point to it.
`void omp_destroy_lock(omp_lock_t *lock)`	Disassociate lock from any locks.
`void omp_set_lock(omp_lock_t *lock)`	Force the executing thread to wait until the lock associated with lock is available. The thread is granted ownership of the lock when it becomes available.
`void omp_unset_lock(omp_lock_t *lock)`	Release executing thread from ownership of lock associated with lock. lock must be initialized via `omp_init_lock()`, and behavior undefined if executing thread does not own the lock associated with lock.
`int omp_test_lock(omp_lock_t *lock);`	Attempt to set lock associated with lock. If successful, return non-zero. lock must be initialized via `omp_init_lock()`.
`void omp_init_nest_lock(omp_nest_lock_t *lock)`	Initialize a unique nested lock and set lock to point to it.
`void omp_destroy_nest_lock(omp_nest_lock_t *lock)`	Disassociate the nested lock lock from any locks.
`void omp_set_nest_lock(omp_nest_lock_t *lock)`	Force the executing thread to wait until the lock associated with lock is available. The thread is granted ownership of the lock when it becomes available
`void omp_unset_nest_lock(omp_nest_lock_t *lock)`	Release executing thread from ownership of lock associated with lock if count is zero. lock must be initialized via `omp_init_nest_lock()`. Behavior is undefined if executing thread does not own the lock associated with lock.
`int omp_test_nest_lock(omp_nest_lock_t *lock)`	Attempt to set lock associated with lock. If successful, return nesting count, otherwise return zero. lock must be initialized via `omp_init_lock()`.

An Interesting Pi Program

Numerical integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i.

Serial PI Program

```
static long num_steps = 100000;
double step;
int main ()
{           int i;      double x, pi, sum = 0.0;

            step = 1.0/(double) num_steps;

            for (i=0;i< num_steps; i++){
                        x = (i+0.5)*step;
                        sum = sum + 4.0/(1.0+x*x);
            }
            pi = step * sum;
}
```

Serial PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
int main ()
{           int i;      double x, pi, sum = 0.0, tdata;

            step = 1.0/(double) num_steps;
            double tdata = omp_get_wtime();
            for (i=0;i< num_steps; i++){
                        x = (i+0.5)*step;
                        sum = sum + 4.0/(1.0+x*x);
            }
            pi = step * sum;
            tdata = omp_get_wtime() - tdata;
            printf(" pi = %f in %f secs\n",pi, tdata);
}
```

The library routine get_omp_wtime() is used to find the elapsed "wall time" for blocks of code

Exercise: the Parallel Pi Program

- Create a parallel version of the pi program using a parallel construct:

  #pragma omp parallel

- Pay close attention to shared versus private variables.
- In addition to a parallel construct, you will need the runtime library routines
  - int omp_get_num_threads();
  - int omp_get_thread_num();
  - double omp_get_wtime();
  - omp_set_num_threads();

Number of threads in the team

Thread ID or rank

Time in Seconds since a fixed point in the past

Request a number of threads in the team

Hints: the Parallel Pi Program

- Use a parallel construct:

  #pragma omp parallel

- The challenge is to:
  - divide loop iterations between threads (use the thread ID and the number of threads).
  - Create an accumulator for each thread to hold partial sums that you can later combine to generate the global sum.

SPMD: Single Program Mulitple Data

- Run the same program on P processing elements where P can be arbitrarily large.
- Use P and the rank … an ID ranging from 0 to (P-1) … to select between a set of tasks and to manage any shared data structures.

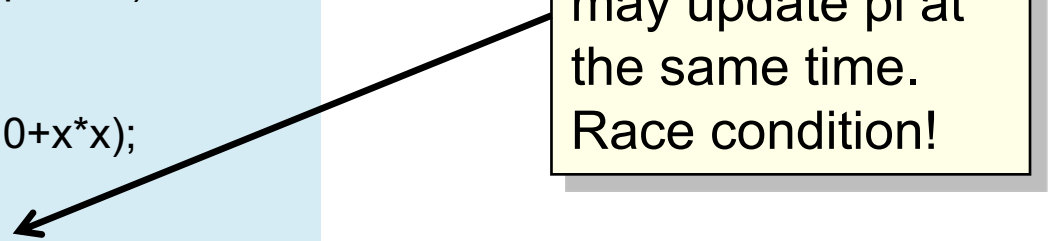
This design pattern is very general and has been used to support most (if not all) the algorithm strategy patterns.

MPI programs almost always use this pattern … it is probably the most commonly used pattern in the history of parallel programming.

Wrong Code: Has Data Race

```
…
int main()
{
    int i; double x, pi, sum = 0.0;
    step = 1.0/(double) num_steps;

#pragma omp parallel private(x, sum)
{
#pragma omp for
    for (i=0; i<=num_steps; i++)
    {
       x=(i+0.5)*step;
       sum=sum+ 4.0/(1.0+x*x);
    }
    pi = pi + step * sum;
}
    printf("pi=%f\n",pi);
    return 0;
}
```



Results*: Use an Array for Local Sum

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

Example: A simple Parallel pi program

```
#include <omp.h>
static long num_steps = 100000;        double step;
#define NUM_THREADS 2
void main ()
{          int i, nthreads;  double pi, sum[NUM_THREADS];
           step = 1.0/(double) num_steps;
           omp_set_num_threads(NUM_THREADS);
   #pragma omp parallel
   {
           int i, id,nthrds;
          double x;
          id = omp_get_thread_num();
          nthrds = omp_get_num_threads();
          if (id == 0)  nthreads = nthrds;
           for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds) {
                    x = (i+0.5)*step;
                    sum[id] += 4.0/(1.0+x*x);
           }
   }
           for(i=0, pi=0.0;i<nthreads;i++)pi += sum[i] * step;
}
```

The false sharing not always holds true. Therefore apply this technique only if the issue occours.

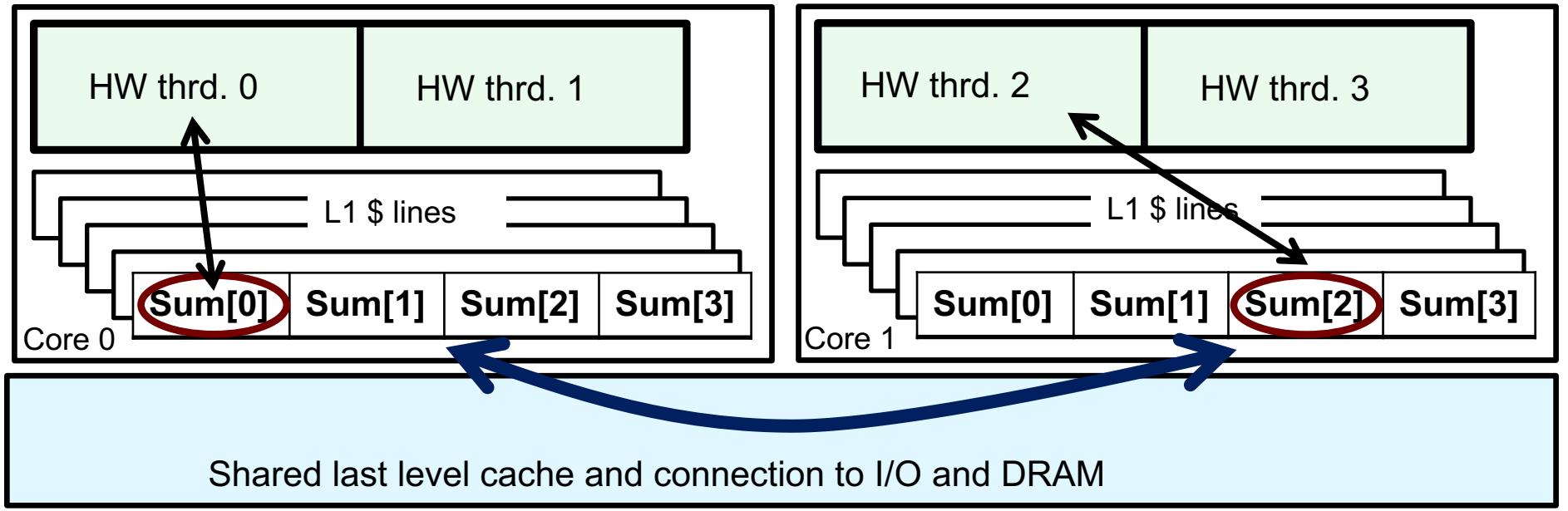
threads	1st SPMD*
1	1.86
2	1.03
3	1.08
4	0.97

*SPMD: Single Program Multiple Data

*Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Why Such Poor Scaling? Has False Sharing

- If independent data elements happen to sit on the same cache line, each update will cause the cache lines to "slosh back and forth" between threads … This is called **"false sharing"**.



- If you promote scalars to an array to support creation of an SPMD program, the array elements are contiguous in memory and hence share cache lines … Results in poor scalability.
- Solution: Pad arrays so elements you use are on distinct cache lines.

Eliminate False Sharing via Padding

```
#include <omp.h>
static long num_steps = 100000;        double step;
#define    PAD      8          // assume 64 byte L1 cache line size
#define NUM_THREADS 2
void main ()
{              int i, nthreads;  double pi, sum[NUM_THREADS][PAD];
               step = 1.0/(double) num_steps;
               omp_set_num_threads(NUM_THREADS);
   #pragma omp parallel
   {           int i, id,nthrds;
            double x;
            id = omp_get_thread_num();
            nthrds = omp_get_num_threads();
            if (id == 0)   nthreads = nthrds;
               for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds) {
                        x = (i+0.5)*step;
                        sum[id][0] += 4.0/(1.0+x*x);
               }
    }
               for(i=0, pi=0.0;i<nthreads;i++) pi += sum[i][0] * step;
}
```

Pad the array so each sum value is in a different cache line

Results*: Pi Program via Padding

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

Example: eliminate False sharing by padding the sum array

```
#include <omp.h>
static long num_steps = 100000;        double step;
#define   PAD    8          // assume 64 byte L1 cache line size
#define NUM_THREADS 2
void main ()
{          int i, nthreads;  double pi, sum[NUM_THREADS][PAD];
           step = 1.0/(double) num_steps;
           omp_set_num_threads(NUM_THREADS);
   #pragma omp parallel
   {
          int i, id,nthrds;
          double x;
          id = omp_get_thread_num();
          nthrds = omp_get_num_threads();
          if (id == 0)   nthreads = nthrds;
           for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds) {
                    x = (i+0.5)*step;
                    sum[id][0] += 4.0/(1.0+x*x);
           }
   }
           for(i=0, pi=0.0;i<nthreads;i++)pi += sum[i][0] * step;
}
```

threads	1st SPMD	1st SPMD padded
1	1.86	1.86
2	1.03	1.01
3	1.08	0.69
4	0.97	0.53

*Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- **Synchronization**
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

Synchronization

- High level synchronization included in the common core
  - Critical
  - Barrier

- The full OpenMP specification has MANY more, such as
  - Atomic

Synchronization is used to impose order constraints and to protect access to shared data

Synchronization: critical

- Mutual exclusion: Only one thread at a time can enter a **critical** region.

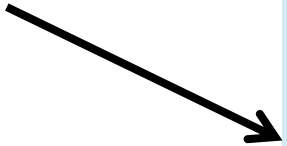
Threads wait their turn – only one at a time calls consume()

```
float  res;                                   critical locks a code segment
#pragma omp parallel
{    float B;   int i, id, nthrds;
     id = omp_get_thread_num();
     nthrds = omp_get_num_threads();
      for(i=id;i<niters;i+=nthrds){
            B =  big_job(i);
#pragma omp critical
            res += consume (B);
     }
}
```

Synchronization: barrier

- Barrier: a point in a program all threads much reach before any threads are allowed to proceed.
- It is a “stand alone” pragma meaning it is not associated with user code … it is an executable statement.
- Barrier makes sure all the shared variables are (explicitly) synchronized.

Threads wait until all threads hit the barrier. Then they can go on.



```
double Arr[8], Brr[8];          int numthrds;
omp_set_num_threads(8)
#pragma omp parallel
{    int id, nthrds;
     id = omp_get_thread_num();
     nthrds = omp_get_num_threads();
     if (id==0) numthrds = nthrds;
     Arr[id] = big_ugly_calc(id, nthrds);
#pragma omp barrier
     Brr[id] = really_big_and_ugly(id, nthrds, Arr);
}
```

atomic

- `# pragma omp atomic [read | write | update | capture]`
- Atomic can protect loads
  - `# pragma omp atomic read`
    `v = x;`
- Atomic can protect stores
  - `# pragma omp atomic write`
    `x = expr;`
- Atomic can protect updates to a storage location (this is the default behavior ... i.e. when you don't provide a clause)
  - `# pragma omp atomic update`
    `x++; or ++x; or x--; or –x;`
    `or x binop= expr; or x = x binop expr;`
    `where binop ∈ {+, -, *, /, &, ^, |, <<, >>}`

atomic

- Atomic can protect the assignment of a value (its capture) **and** an associated update operation:
  - `# pragma omp atomic capture`
    `statement or structured block`
- Where the statement is one of the following forms:
  - `v=x++; v=++x; v=x--; v= --x; v=x binop expr;`
- Where the structured block is one of the following forms:
  - `{v=x; x binop = expr;}`
  - `{x binop = expr; v=x;}`
  - `{v=x; x = x binop expr;}`
  - `{x = x binop expr; v=x;}`
  - `{v=x; x++;}`
  - `{v=x; ++x}`
  - `{++x; v=x}`
  - `{x++; v=x;}`
  - `{v=x; x--;}`
  - `{v=x; --x;}`
  - `{--x; v=x;}`
  - `{x--; v=x;}`

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- **Parallel Loops**
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

The Worksharing-Loop Constructs

- The worksharing-loop construct splits up loop iterations among the threads in a team

```
#pragma omp parallel
{
#pragma omp for
        for (I=0;I<N;I++){
                NEAT_STUFF(I);
        }
}
```

Loop construct name:

- C/C++: for
- Fortran: do

The loop control index I is made "private" to each thread by default.

Threads wait here until all threads are finished with the parallel loop before any proceed past the end of the loop

Worksharing-Loop Constructs
A Motivating Example

Sequential code

```
for(i=0;i<N;i++)   { a[i] = a[i] + b[i];}
```

OpenMP parallel region

```
#pragma omp parallel
{
        int id, i, Nthrds, istart, iend;
        id = omp_get_thread_num();
        Nthrds = omp_get_num_threads();
        istart = id * N / Nthrds;
        iend = (id+1) * N / Nthrds;
        if (id == Nthrds-1) iend = N;
        for(i=istart;i<iend;i++)   { a[i] = a[i] + b[i];}
}
```

OpenMP parallel region and a worksharing for construct

```
#pragma omp parallel
#pragma omp for
        for(i=0;i<N;i++)   { a[i] = a[i] + b[i];}
```

Worksharing-Loop Constructs: the schedule Clause

- The schedule clause affects how loop iterations are mapped onto threads
  - schedule(static [,chunk])
    - Deal-out blocks of iterations of size “chunk” to each thread.
  - schedule(dynamic[,chunk])
    - Each thread grabs “chunk” iterations off a queue until all iterations have been handled.

Schedule Clause	When To Use
STATIC	**Pre-determined and predictable by the programmer**
DYNAMIC	**Unpredictable, highly variable work per iteration**

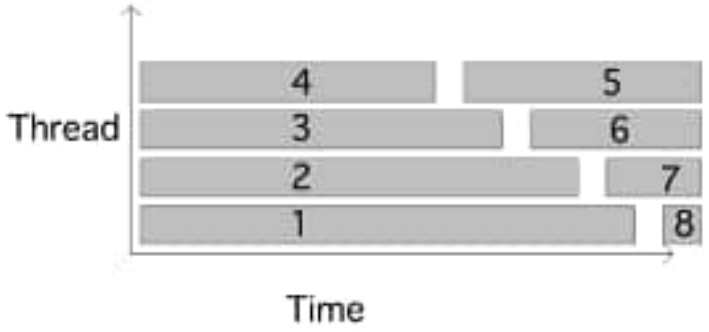
Least work at runtime : scheduling done at compile-time

Most work at runtime : complex scheduling logic used at run-time

Loop schedules

- Default: static scheduling of iterations.
  Very efficient. Good if all iterations take the same amount of time.
  `schedule(static)`
- Other possibility: dynamic.
  Runtime overhead; better if iterations do not take the same amount of time.
  `schedule(dynamic)`

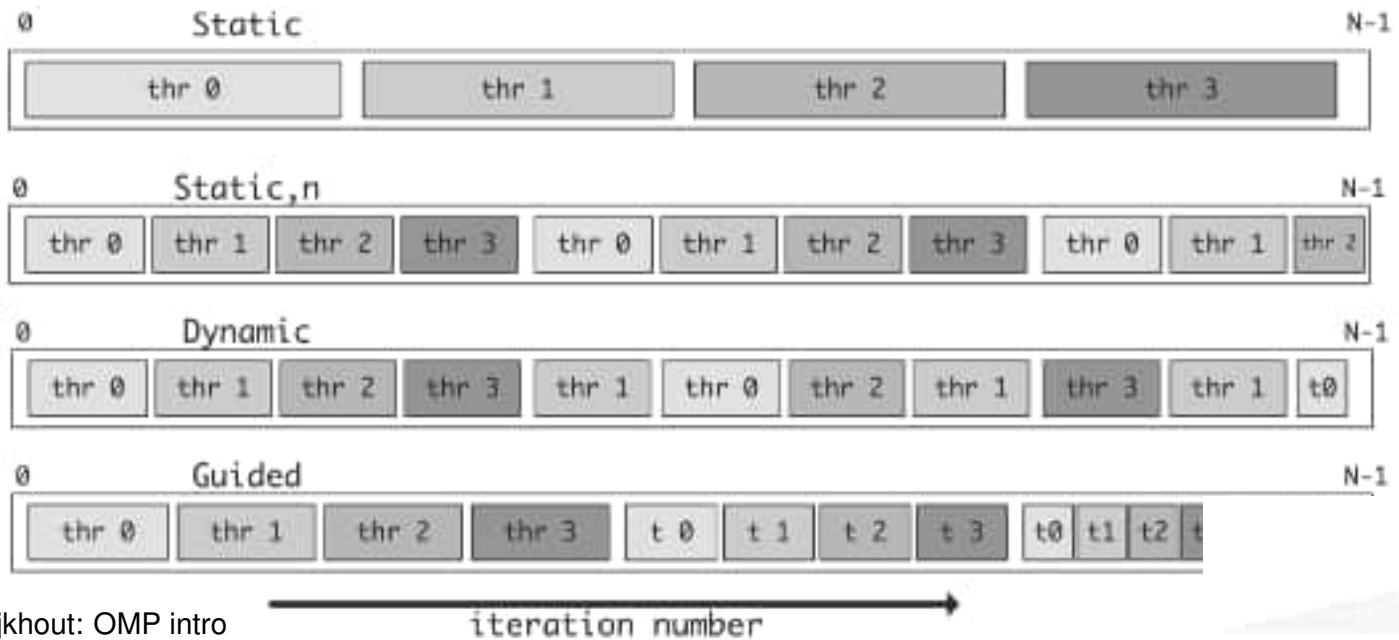
Four threads, 8 tasks of decreasing size
dynamic schedule is better:



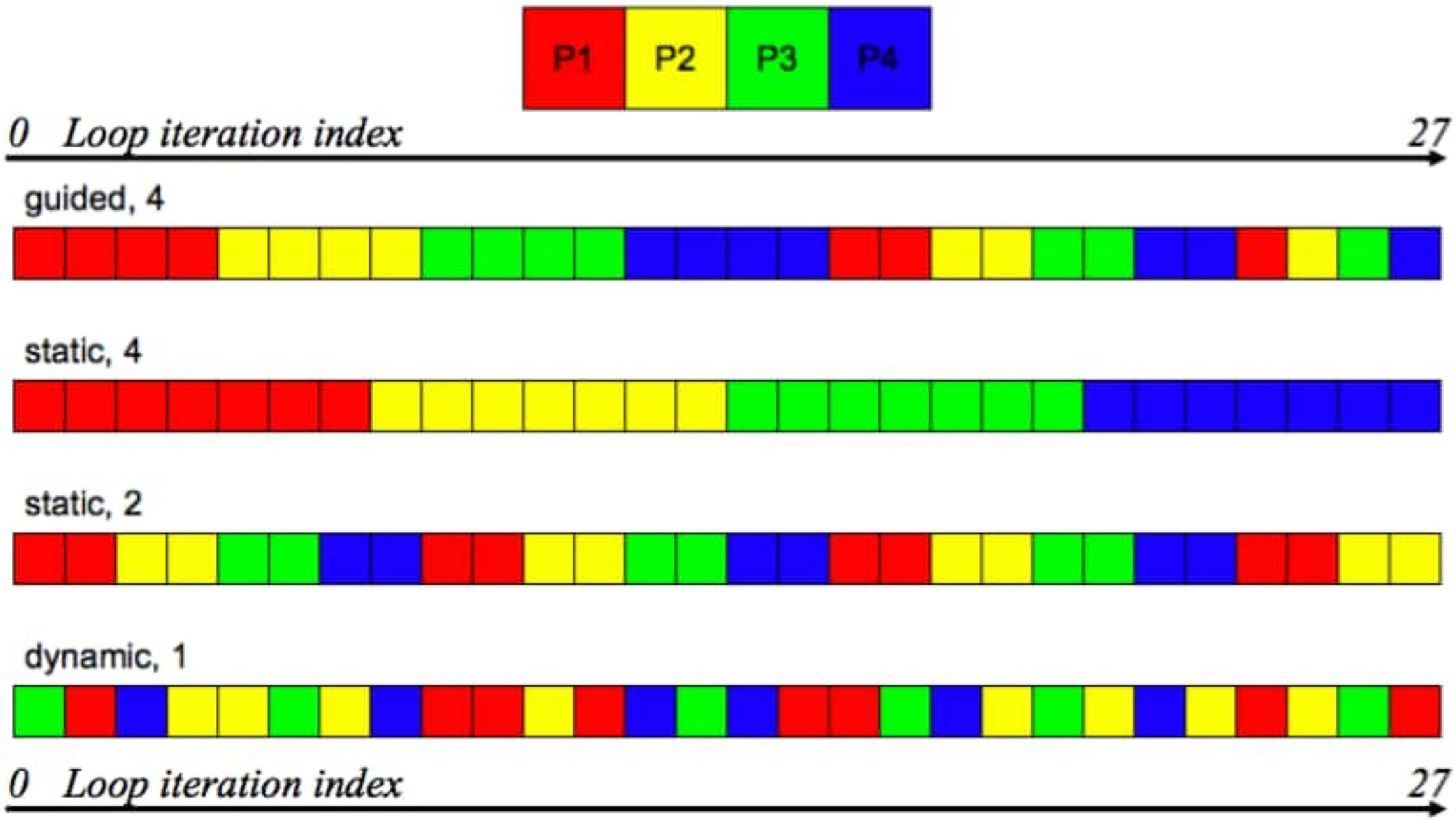
Chunk size

With N iterations and t threads:

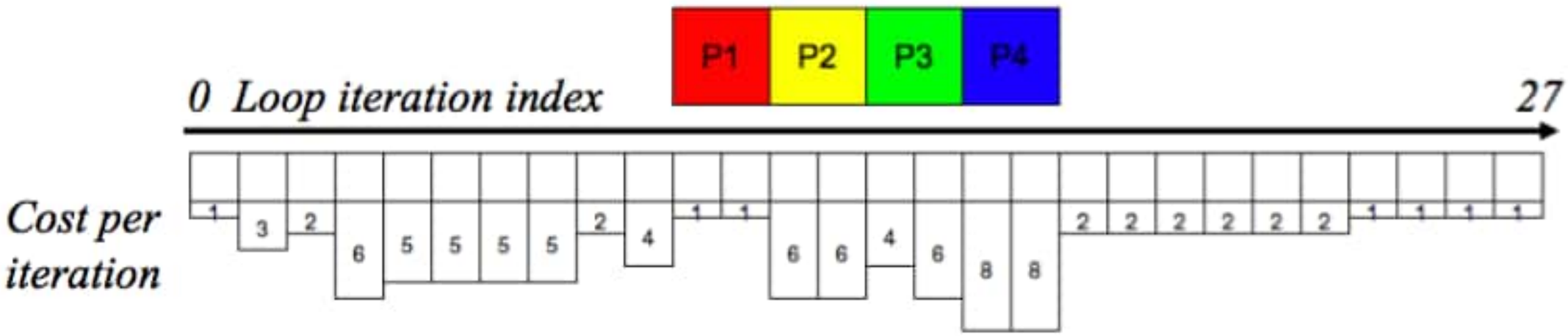
- Static: each thread gets N/t iterations.
  explicit chunk size: `schedule(static,123)`
- Dynamic: each thread gets 1 iteration at a time
  explicit chunk size: `schedule(dynamic,45)`
- Help from OpenMP:
  guided schedule uses decreasing chunk size (with optional minimum chunk):
  `schedule(guided,6)`

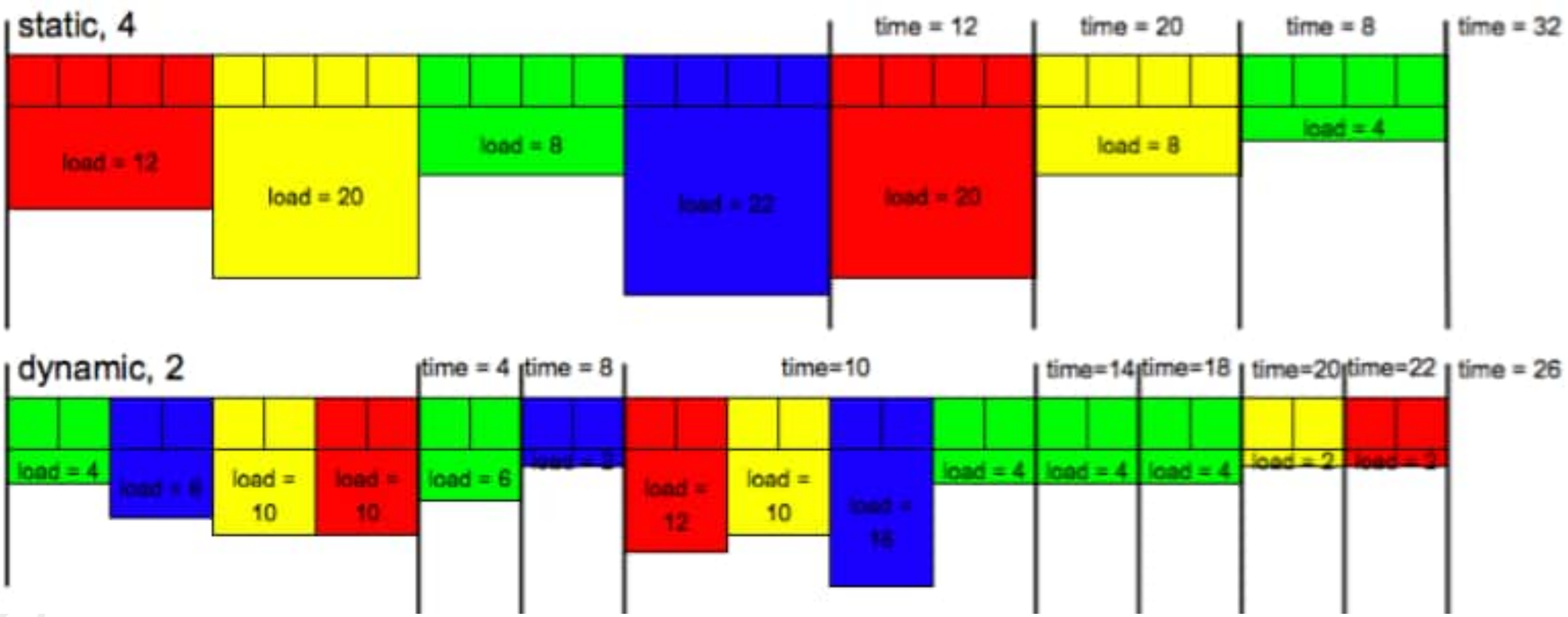


for loop scheduling



for loop scheduling



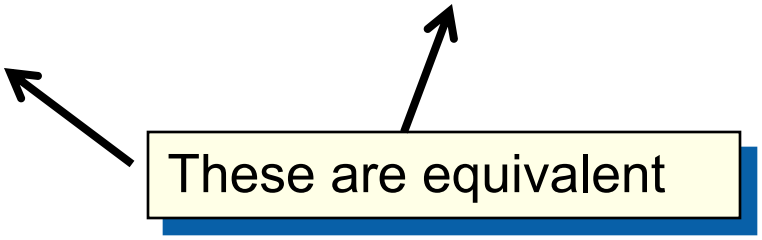


Combined Parallel/Worksharing Construct

- OpenMP shortcut: Put the "parallel" and the worksharing directive on the same line

```
 double  res[MAX];  int i;
#pragma omp parallel
{
    #pragma omp for
    for (i=0;i< MAX; i++) {
        res[i] = huge();
    }
}
```

```
 double  res[MAX];  int i;
#pragma omp parallel for
    for (i=0;i< MAX; i++) {
        res[i] = huge();
    }
```



The Reduction Clause

Serial

```
double ave=0.0, A[max]; int i;
for (i=0; i< max; i++) {
    ave + = A[i];
}
ave = ave/max;
```

Parallel

```
double ave=0.0, A[max]; int i;
#pragma omp parallel for reduction (+:ave)
  for (i=0; i< max; i++) {
      ave + = A[i];
  }
 ave = ave/max;
```

- **Common accumulation pattern, with loop dependencies in serial code.**
- **Syntax: Reduction (operator : list)**
- **Reduces list of variables into one, using operator**
- **Reduced variables must be shared variables**
- **Allowed Operators in Common Core:**
  - **arithmetic: + - ***
  - **math: max min**
- **A local copy of each list variable is made and initialized depending on the "op" (e.g. 0 for +, 1 for *)**
- **Each thread does its local accumulation first, then a global reduction is done at the end**

Reduction Operands/Initial-values

- Many different associative operands can be used with reduction:
- Initial values are the ones that make sense mathematically.

Operator	Initial value
+	0
*	1
-	0
min	Largest pos. number
max	Most neg. number

Exercise: Pi with Loops and a Reduction

- Go back to the serial Pi program and parallelize it with a loop construct
- Your goal is to minimize the number of changes made to the serial program.

```
#pragma omp parallel
#pragma omp for
#pragma omp parallel for
#pragma omp for reduction(op:list)
#pragma omp critical
int omp_get_num_threads();
int omp_get_thread_num();
double omp_get_wtime();
```

Remember: OpenMP makes the loop control index in a loop workshare construct private for you … you don't need to do this yourself

Pi with a Loop and a Reduction

```
#include <omp.h>
static long num_steps = 100000;        double step;
void main ()
{    int i;   double x, pi, sum = 0.0;
      step = 1.0/(double) num_steps;
      #pragma omp parallel
      {
          double x;
         #pragma omp for reduction(+:sum)
               for (i=0;i< num_steps; i++) {
                        x = (i+0.5)*step;
                        sum = sum + 4.0/(1.0+x*x);
               }
       }
             pi = step * sum;
}
```

Create a team of threads … without a parallel construct, you'll never have more than one thread

Create a scalar local to each thread to hold value of x at the center of each interval

Break up loop iterations and assign them to threads … setting up a reduction into sum. Note … the loop index is local to a thread by default.

Results*: Pi with a Loop and a Reduction

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

Example: Pi with a loop

```
#include <omp.h>
static long num_steps = 100000;
void main ()
{   int i;          double x, pi, sum =
    step = 1.0/(double) num_steps;
    #pragma omp parallel
    {
      double x;
      #pragma omp for reduction(+:sum)
        for (i=0;i< num_steps; i++){
            x = (i+0.5)*step;
            sum = sum + 4.0/(1.0+x*x);
        }
    }
        pi = step * sum;
}
```

threads	1st SPMD	1st SPMD padded	SPMD critical	PI Loop and reduction
1	1.86	1.86	1.87	1.91
2	1.03	1.01	1.00	1.02
3	1.08	0.69	0.68	0.80
4	0.97	0.53	0.53	0.68

*Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Autoparallelization

icc -qopenmp -parallel pi.c

If -qopt_report=5

```
#include <stdio.h>
#include <omp.h>

int main (int argc, char** argv){
    static long num_steps = 1000000000;
    double step;
    double start, stop;

    int i; double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;
    start = omp_get_wtime();
    for (i=0;i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
        pi = step * sum;
    }
    stop = omp_get_wtime();
    printf("Pi is: %.20f\nIn %f seconds\n",pi, st
}
```

```
LOOP BEGIN at pi.c(13,5)
   remark #17109: LOOP WAS AUTO-PARALLELIZED
   remark #17101: parallel loop shared={ } private={ } firstprivate={ x i } lastprivate={ } firstla
   remark #15305: vectorization support: vector length 2
   remark #15399: vectorization support: unroll factor set to 4
   remark #15309: vectorization support: normalized vectorization overhead 0.108
   remark #15300: LOOP WAS VECTORIZED
   remark #15475: --- begin vector cost summary ---
   remark #15476: scalar cost: 46
   remark #15477: vector cost: 25.500
   remark #15478: estimated potential speedup: 1.800
   remark #15486: divides: 1
   remark #15487: type converts: 1
   remark #15488: --- end vector cost summary ---
   remark #25015: Estimate of max trip count of loop=125000000
LOOP END

LOOP BEGIN at pi.c(13,5)
   remark #15305: vectorization support: vector length 2
   remark #15399: vectorization support: unroll factor set to 4
   remark #15309: vectorization support: normalized vectorization overhead 0.108
   remark #15300: LOOP WAS VECTORIZED
   remark #15475: --- begin vector cost summary ---
   remark #15476: scalar cost: 46
   remark #15477: vector cost: 25.500
   remark #15478: estimated potential speedup: 1.800
   remark #15486: divides: 1
   remark #15487: type converts: 1
   remark #15488: --- end vector cost summary ---
   remark #25015: Estimate of max trip count of loop=125000000
LOOP END

LOOP BEGIN at pi.c(13,5)
<Remainder loop for vectorization>
   remark #15335: remainder loop was not vectorized: vectorization possible but seems inefficient.
   remark #15305: vectorization support: vector length 2
   remark #15309: vectorization support: normalized vectorization overhead 0.627
   remark #25015: Estimate of max trip count of loop=1000000000
LOOP END
```

… reduction {sum}

The nowait Clause

- Barriers are really expensive. You need to understand when they are implied and how to skip them when its safe to do so.

```
double A[big], B[big], C[big];

#pragma omp parallel
{
        int id=omp_get_thread_num();
        A[id] = big_calc1(id);
#pragma omp barrier
#pragma omp for
        for(i=0;i<N;i++){C[i]=big_calc3(i,A);}
#pragma omp for nowait
        for(i=0;i<N;i++){ B[i]=big_calc2(C,  i); }
        A[id] = big_calc4(id);
}
```

implicit barrier at the end of a for worksharing construct

no implicit barrier due to nowait

implicit barrier at the end of a parallel region

Nested loops

- For perfectly nested rectangular loops we can parallelize multiple loops in the nest with the collapse clause:

```
#pragma omp parallel for collapse(2)
for (int i=0; i<N; i++) {
  for (int j=0; j<M; j++) {
          .....
  }
}
```

Number of loops to be parallelized, counting from the outside

- Will form a single loop of length NxM and then parallelize that.
- Useful if N is O(no. of threads) so parallelizing the outer loop makes balancing the load difficult.

Performance Tips

- Is there enough work to amortize overheads?
  - May not be worthwhile for very small loops (if clause can control this)
  - Might be overcome by choosing different loop, rewriting loop nest or collapsing loop nest
- Best choice of schedule might change with system, problem size
  - Experimentation may be needed
- Minimize synchronization
  - Use nowait where possible
- Locality
  - Most large systems are NUMA
  - Be prepared to modify your loop nests
  - Change loop order to get better cache behavior
- If performance is bad, look for false sharing
  - We talk about this in part 2 of the tutorial
  - Occurs frequently, performance degradation can be catastrophic

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- **Data Environment**
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- Recap, Q&A

OpenMP Data Environment

- Most variables are shared by default
  - Fortran: common blocks, SAVE variables, module variables
  - C/C++: file scope variables, static variables
  - Both: dynamically allocated variables (ALLOCATE, malloc, new)
- Some variables are private by default
  - Certain loop indices
  - Stack variables in subprograms (Fortran) or functions (C) called from parallel regions
  - Automatic (local) variables within a statement block

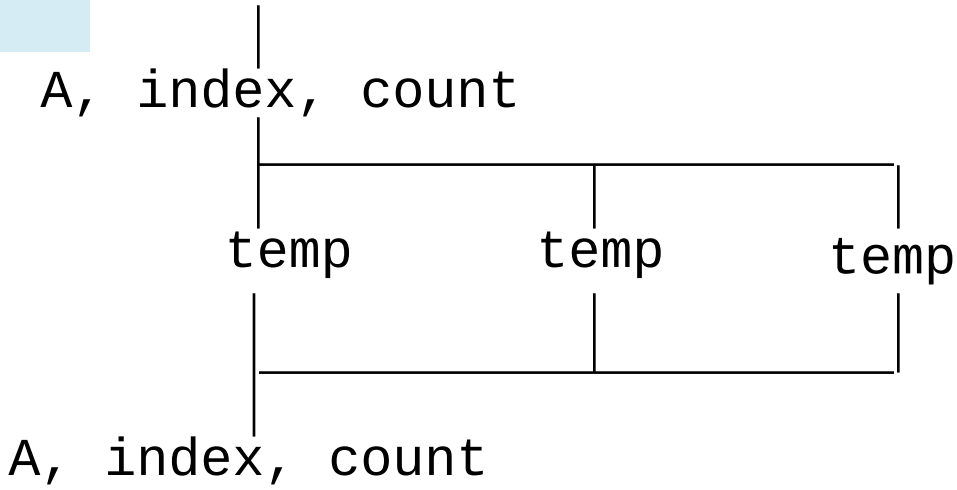
A Data Sharing Example

```
double A[10];
    int main() {
    int index[10];
   #pragma omp parallel
          work(index);
    printf("%d\n", index[0]);
  }
```

```
extern double A[10];
void work(int *index) {
  double temp[10];
  static int count;
  ...
}
```

A, index and count are shared by all threads.

temp is local to each thread



Shared memory problems

Race condition: simultaneous update of shared data:

process 1: `I=I+2`
process 2: `I=I+3`

Results can be indeterminate:

scenario 1.		scenario 2.		scenario 3.	
$\texttt{I} = 0$					
read $\texttt{I} = 0$	read $\texttt{I} = 0$	read $\texttt{I} = 0$	read $\texttt{I} = 0$	read $\texttt{I} = 0$	
compute $\texttt{I} = 2$	compute $\texttt{I} = 3$	compute $\texttt{I} = 2$	compute $\texttt{I} = 3$	compute $\texttt{I} = 2$	
write $\texttt{I} = 2$			write $\texttt{I} = 3$	write $\texttt{I} = 2$	
	write $\texttt{I} = 3$	write $\texttt{I} = 2$			read $\texttt{I} = 2$
					compute $\texttt{I} = 5$
					write $\texttt{I} = 5$
$\texttt{I} = 3$		$\texttt{I} = 2$		$\texttt{I} = 5$	

Data sharing: Changing storage attributes

- One can selectively change storage attributes for constructs using the following clauses: (note: list is a comma-separated list of variables)
  - shared(list)
  - private(list)
  - firstprivate(list)

These clauses apply to the OpenMP construct NOT to the entire region.

- These can be used on parallel and for constructs … other than shared which can only be used on a parallel construct

- Force the programmer to explicitly define storage attributes
  - default (none)

default() can be used on parallel constructs

Data Sharing: private Clause

- private(var) creates a new local copy of var for each thread.
  - The value of the private copies is uninitialized
  - The storage of the private copy will be on the each thread's stack memory, and is unassociated with the original variable
  - The value of the original variable is unchanged after the region

```
void wrong() {
      int tmp = 0;
#pragma omp parallel for private(tmp)
      for (int j = 0; j < 1000; ++j)
            tmp += j;
      printf("%d\n", tmp);
}
```

When you need to reference the variable tmp that exists prior to the construct, we call it the **original variable**.

tmp was not initialized

tmp is 0 here

Data Sharing: Private Clause When is the Original Variable Valid?

- The original variable's value is unspecified if it is referenced outside of the construct
  - Implementations may reference the original variable or a copy ….. a dangerous programming practice!
  - For example, consider what would happen if the compiler inlined work()?

```
int tmp;
void danger() {
      tmp = 0;
#pragma omp parallel private(tmp)
      work();
     printf("%d\n", tmp);
}
```

tmp has unspecified value

```
extern int tmp;
void work() {
      tmp = 5;
}
```

unspecified which copy of tmp

The Firstprivate Clause

- Initializes the variables in the list with the value of the shared variable when they first enter the construct
- C++ objects are copy-constructed

```
incr = 0;
#pragma omp parallel for firstprivate(incr)
for (i = 0; i <= MAX; i++) {
        if ((i%2)==0) incr++;
        A[i] = incr;
}
```

Each thread gets its own copy of incr with an initial value of 0

Data Sharing:
A Data Environment Test

- Consider this example of PRIVATE and FIRSTPRIVATE

```
variables:  A = 1,B = 1, C = 1
#pragma omp parallel private(B)  firstprivate(C)
```

- Are A,B,C private to each thread or shared inside the parallel region?
- What are their initial values inside and values after the parallel region?

Data Sharing:
A Data Environment Test

- Consider this example of PRIVATE and FIRSTPRIVATE

```
variables:  A = 1,B = 1, C = 1
#pragma omp parallel private(B)  firstprivate(C)
```

- Are A,B,C private to each thread or shared inside the parallel region?
- What are their initial values inside and values after the parallel region?

Inside this parallel region ...

- "A" is shared by all threads; equals 1
- "B" and "C" are private to each thread.
  - B's initial value is undefined
  - C's initial value equals 1

Following the parallel region ...

- B and C revert to their original values of 1
- A is either 1 or the value it was set to inside the parallel region

private variables in/out

- Use `firstprivate` to declare private variables that are initialized with the main thread's value of the variables

- Use `lastprivate` to declare private variables whose values are copied back out to main thread's variables by the thread that executes the last iteration of a parallel for loop, or the thread that executes the last parallel section

- These are special cases of private, and are useful to bring values in and out from the parallel section of code.

Data Sharing: default Clause

- **default(none)**: Forces you to define the storage attributes for variables that appear inside the static extent of the construct … if you fail the compiler will complain. Good programming practice!
- You can put the default clause on parallel and parallel + workshare constructs.

```
#include <omp.h>
int main()
{
    int i, j=5;      double x=1.0, y=42.0;
    #pragma omp parallel for default(none) reduction(*:x)
    for (i=0;i<N;i++){
        for(j=0; j<3; j++)
            x+= foobar(i, j, y);
    }
    printf(" x is %f\n",(float)x);
}
```

The static extent is the code in the compilation unit that contains the construct.

The compiler would complain about j and y, which is important since you don't want j to be shared

The full OpenMP specification has other versions of the default clause, but they are not used very often so we skip them in the common core

Performance and Correctness Tips

- There is one version of shared data
  - Keeping data shared reduces overall memory consumption
- Private data is stored locally, so use of private variables can increase efficiency
  - Avoids false sharing
  - May make it easier to parallelize loops
  - But private data is no longer available after parallel regions ends

- It is an error if multiple threads update the same variable at the same time (a data race)
- It is a good idea to use “default none” while testing code
- Putting code into a subroutine / function can make it easier to write code with many private variables
  - Local / automatic data in a procedure is private by default

UNIVERSITÀ
DEGLI STUDI
FIRENZE

OpenMP memory model and synchronization

OpenMP memory model

- In the shared memory model of OpenMP all threads share an address space ... but what they actually see at a given point in time may be complicated: a variable residing in shared memory may be in the cache of several CPUs/cores.

- A memory model is defined in terms of:

  - Coherence: Behavior of the memory system when a single address is accessed by multiple threads.

  - Consistency: Orderings of reads, writes, or synchronizations (RWS) with various addresses and by multiple threads.

OpenMP memory model

- In the shared memory model of OpenMP all threads share an address space ... but what they actually see at a given point in time may be complicated: a variable residing in shared memory may be in the cache of several CPUs/cores.

At a given point in time, the “private view” seen by a thread may be different from the view in shared memory.

In fact, there are several re-orderings from the original source code:
- Compiler re-orders program order to the code order
- Machine re-orders code order to the memory commit order

Consistency

- Sequential Consistency:
  - In a multi-processor, ops (R, W, S) are sequentially consistent if:
    - They remain in program order for each processor.
    - They are seen to be in the same overall order by each of the other processors.
  - Program order = code order = commit order
- Relaxed consistency:
  - Remove some of the ordering constraints for memory ops (R, W, S).

OpenMP consistency

- OpenMP has a relaxed consistency:
  - S ops must be in sequential order across threads.
  - Can not reorder S ops with R or W ops on the same addresses on the same thread
  - The S operation provided by OpenMP is `flush`

OpenMP consistency

Relaxed consistency means that memory updates made by one CPU may not be immediately visible to another CPU

- Data can be in registers
- Data can be in cache
  (cache coherence protocol is slow or non-existent)

Therefore, the updated value of a shared variable that was set by a thread may not be available to another

The `flush` construct flushes shared variables from local storage (registers, cache) to shared memory

- The S operation provided by OpenMP is `flush`

OpenMP consistency

Relaxed consistency means that memory updates made by one CPU may not be immediately visible to another CPU

- Data can be in registers
- Data can be in cache
  (cache coherence protocol is slow or non-existent)

Therefore, the updated value of a shared variable that was set by a thread may not be available to another

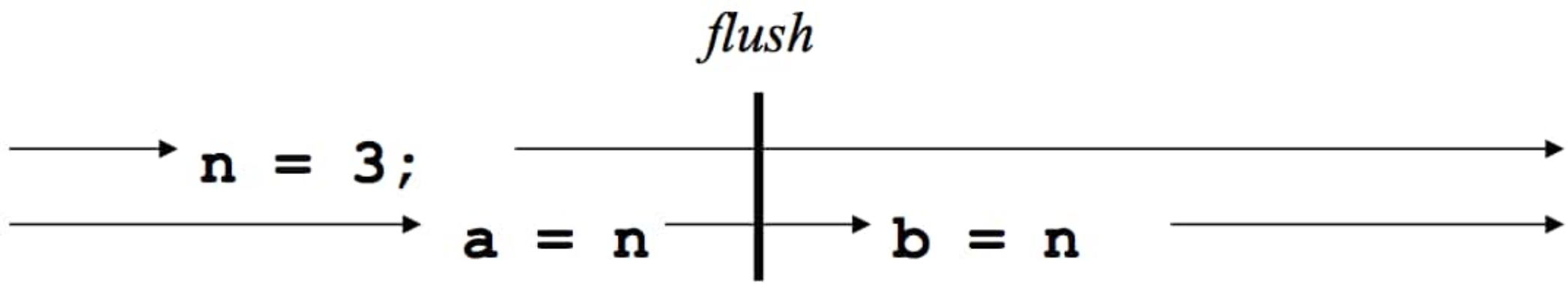
The `flush` construct flushes shared variables from local storage (registers, cache) to shared memory

```
double A;
A = compute();
#pragma omp flush(A); // flush to memory to make sure other
                      // threads can pick up the right value
```

Implicit `flush`

- An OpenMP `flush` is automatically performed at:
  - Entry and exit of `parallel` and `critical` and `atomic` (only variable being atomically updated)
    - unless `nowait` is specified
  - Exit of `for`
  - Exit of `sections`
  - Exit of `single`
  - Barriers
  - When setting/unsetting/testing locks after acquisition

- the `flush` operation **does not actually synchronize** different threads. It just ensures that a thread's values are made consistent with main memory.



- b = 3, but there is no guarantee that *a* will be 3

flush

- Defines a sequence point at which a thread is guaranteed to see a consistent view of memory with respect to the “flush set”.
- The `flush` set is:
  - “all thread visible variables” for a `flush` construct without an argument list.
  - a list of variables when the `flush(list)` construct is used.
- The action of `flush` is to guarantee that:
  - All R,W ops that overlap the flush set and occur prior to the flush complete before the flush executes
  - All R,W ops that overlap the flush set and occur after the flush don’t execute until after the flush.
  - Flushes with overlapping flush sets can not be reordered.
- Flush forces data to be updated in memory so other threads see the most recent value.

- A `flush` construct with a list applies the flush operation to the items in the list,and does not return until the operation is complete for all specified list items.
  - If a pointer is present in the list, the pointer itself is flushed, not the object to which the pointer refers

- A `flush` construct without a list, executed on a given thread, operates as if the whole thread-visible data state of the program is flushed.

Incorrect example:

a = b = 0

thread 1	*thread 2*
b = 1	**a = 1**
flush(**b**)	*flush*(**a**)
flush(**a**)	*flush*(**b**)
if (a == 0) then	**if (b == 0) then**
critical section	*critical section*
end if	**end if**

Correct example:

a = b = 0

thread 1	*thread 2*
b = 1	**a = 1**
flush(**a,b**)	*flush*(**a,b**)
if (a == 0) then	**if (b == 0) then**
critical section	*critical section*
end if	**end if**

Exercise: Mandelbrot Set Area

- The supplied program (mandel.c) computes the area of a Mandelbrot set.
- The program has been parallelized with OpenMP, but we were lazy and didn't do it right.
- Find and fix the errors (hint … the problem is with the data environment).
- Once you have a working version, try to optimize the program.
  - Try different schedules on the parallel loop.
  - Try different mechanisms to support mutual exclusion … do the efficiencies change?

The Mandelbrot Area Program

```
#include <omp.h>
# define NPOINTS 1000
# define MXITR 1000
struct d_complex{
  double r;     double i;
};
void testpoint(struct d_complex);
struct d_complex c;
int numoutside = 0;

int main(){
  int i, j;
  double area, error, eps  = 1.0e-5;
#pragma omp parallel for default(shared) private(c, j) \
   firstprivate(eps)
  for (i=0; i<NPOINTS; i++) {
    for (j=0; j<NPOINTS; j++) {
      c.r = -2.0+2.5*(double)(i)/(double)(NPOINTS)+eps;
      c.i = 1.125*(double)(j)/(double)(NPOINTS)+eps;
      testpoint(c);
    }
  }
area=2.0*2.5*1.125*(double)(NPOINTS*NPOINTS-
numoutside)/(double)(NPOINTS*NPOINTS);
  error=area/(double)NPOINTS;
}
```

```
void testpoint(struct  d_complex c){
struct d_complex z;
      int iter;
      double temp;

      z=c;
      for (iter=0; iter<MXITR; iter++){
        temp = (z.r*z.r)-(z.i*z.i)+c.r;
        z.i = z.r*z.i*2+c.i;
        z.r = temp;
        if ((z.r*z.r+z.i*z.i)>4.0) {
        #pragma omp critical
          numoutside++;
          break;
        }
      }
}
```

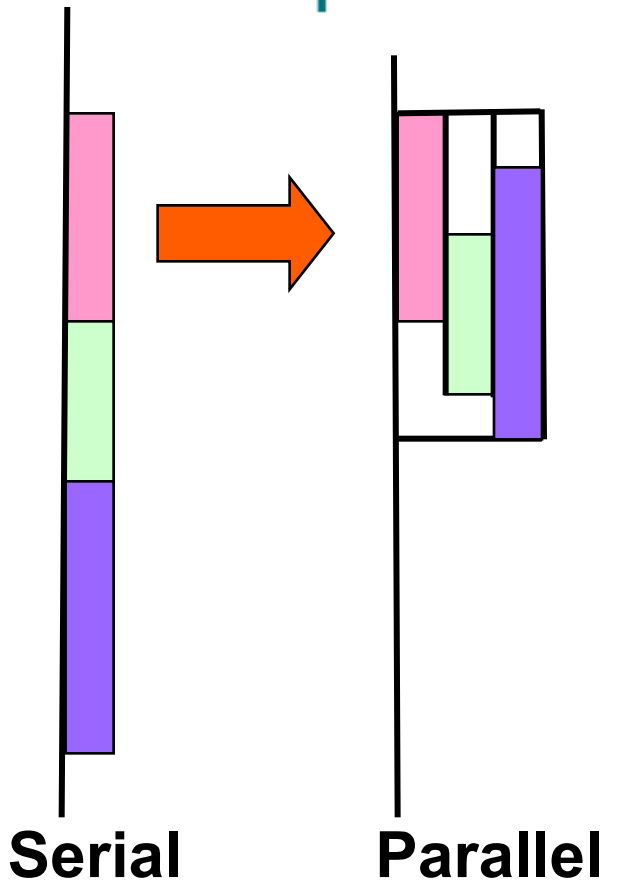
- eps was not initialized
- Protect updates of numoutside
- Which value of c does testpoint() see? Global or private?

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- **Irregular Parallelism and Tasks**
- OpenMP and Performance
- Recap, Q&A

What are OpenMP Tasks?

- Task construct: a structured block of code + a data environment
- Inside a parallel region, a thread encountering a task construct will package up the code block and its data for execution
- The task is executed immediately, or deferred for later execution.
- Tasks can be nested: i.e. a task may itself generate tasks.



A common Pattern is to have one thread create the tasks while the other threads wait at a barrier and execute the tasks

list traversal

- When we first created OpenMP, we focused on common use cases in HPC … Fortran arrays processed over “regular” loops.
- Recursion and “pointer chasing” were so far removed from our Fortan focus that we didn’t even consider more general structures.
- Hence, even a simple list traversal is exceedingly difficult with the original versions of OpenMP.

```
p=head;
while (p) {
   process(p);
   p = p->next;
}
```

Linked lists without tasks

- See the file Linked_omp25.c

```
while (p != NULL) {
    p = p->next;
     count++;
}
p = head;
for(i=0; i<count; i++) {
     parr[i] = p;
     p = p->next;
  }
#pragma omp parallel
{
    #pragma omp for schedule(static,1)
    for(i=0; i<count; i++)
       processwork(parr[i]);
}
```

Count number of items in the linked list

Copy pointer to each node into an array

Process nodes in parallel with a for loop

	Default schedule	Static,1
One Thread	48 seconds	45 seconds
Two Threads	39 seconds	28 seconds

Results on an Intel dual core 1.83 GHz CPU, Intel IA-32 compiler 10.1 build 2

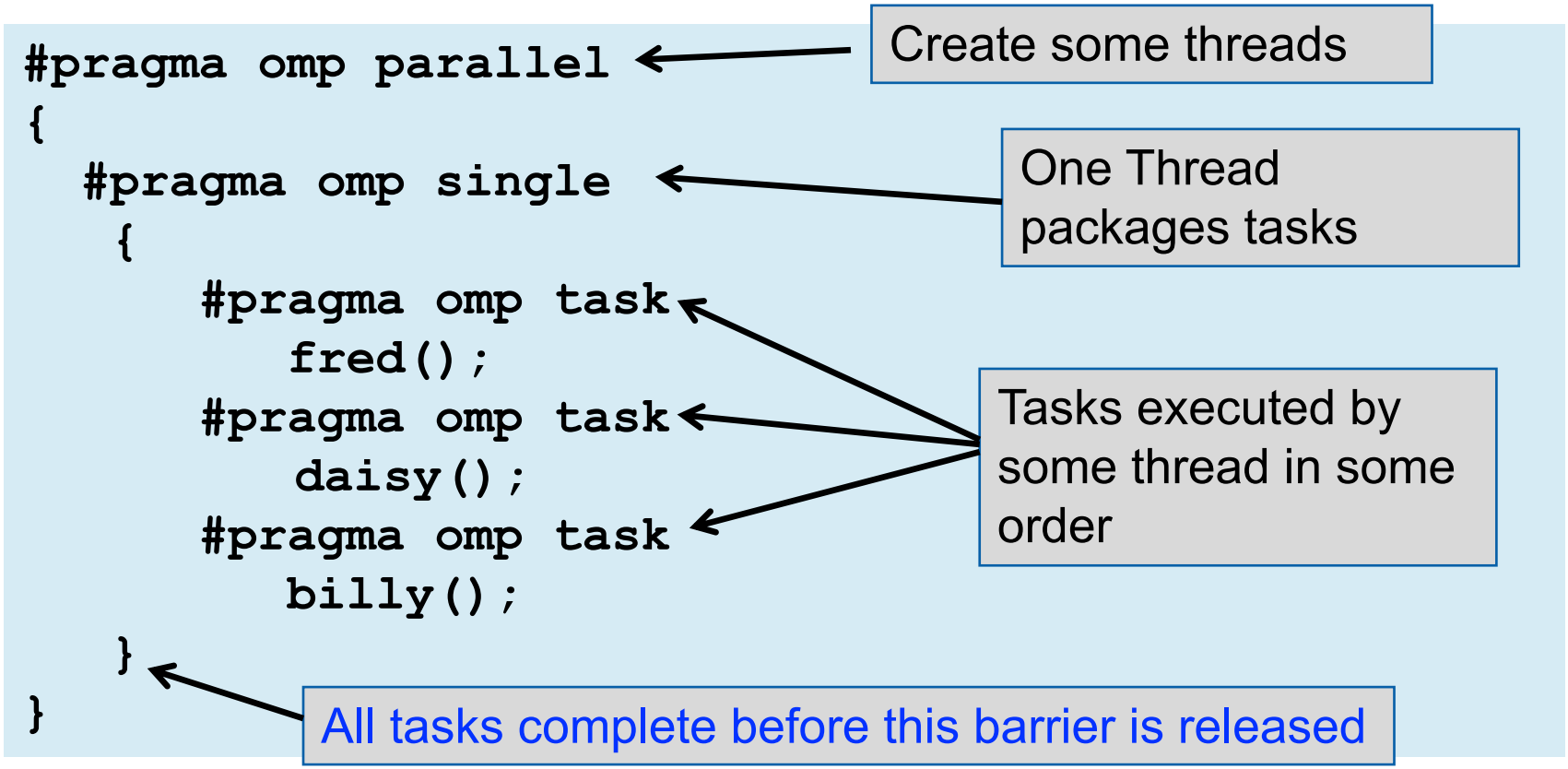
Conclusion

- We were able to parallelize the linked list traversal … but it was ugly and required multiple passes over the data.
- To move beyond its roots in the array based world of scientific computing, we needed to support more general data structures and loops beyond basic for loops.
- To do this, we added tasks in OpenMP 3.0

Task Directive

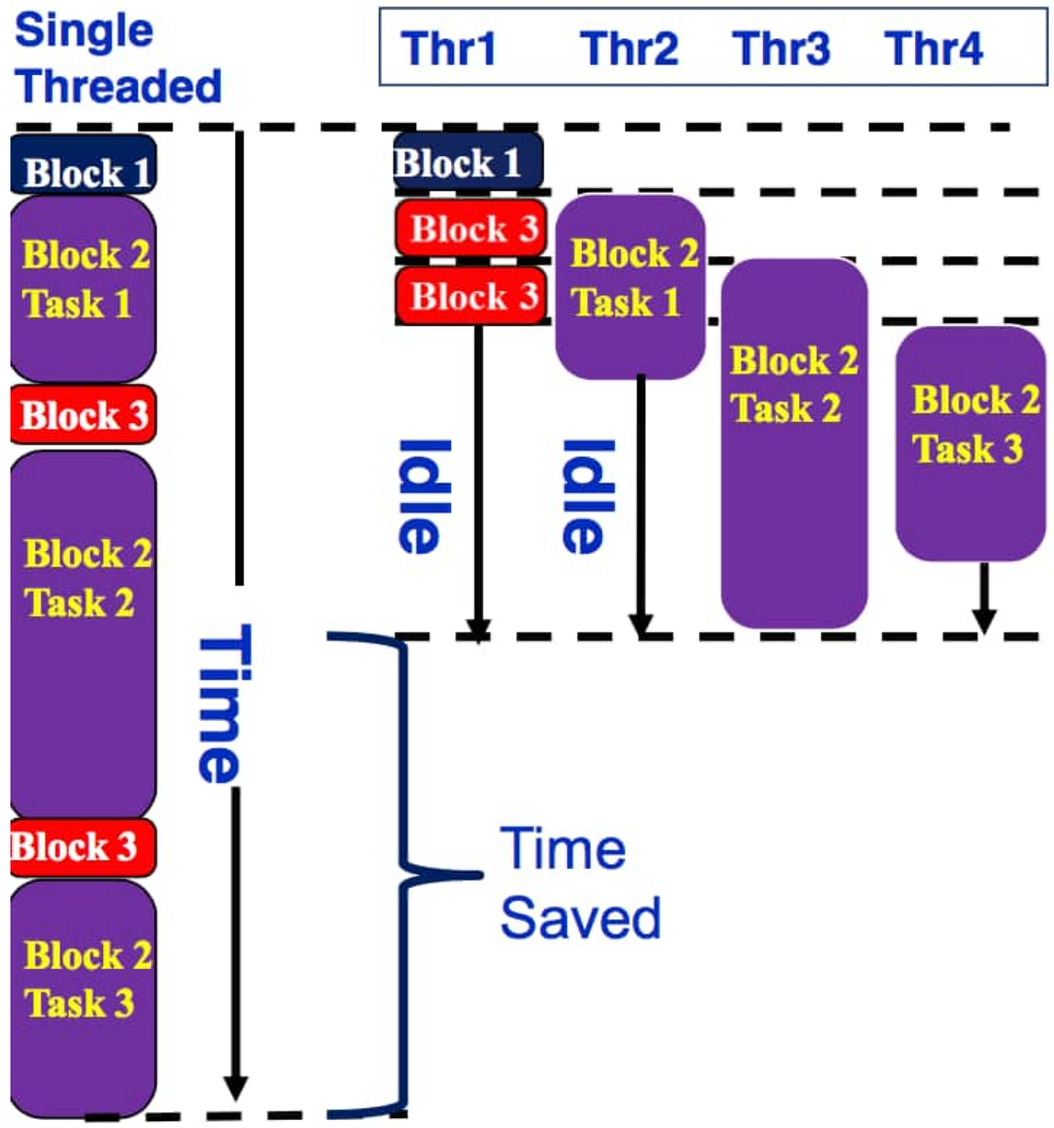
`#pragma omp task` *[clauses]*

structured-block



task motivations

```
#pragma omp parallel
{
#pragma omp single
   { //block 1
     node * p = head;
     while (p) { // block 2
#pragma omp task
        process(p);
        p = p->next; //block 3
     } // end while
   } // end single block
} // end parallel block
```



for and task

- ```
  #pragma omp parallel
  {
  #pragma omp for private(p)
    for ( int i =0; i <numlists ; i++) {
      p = listheads[ i ] ;
      while( p ) {
  #pragma omp task
        process(p)
        p = next( p ) ;
      } // end while
    } // end for
  } // end parallel
  ```

- Example – parallel pointer chasing on multiple lists using tasks (nested parallelism)
  ```

Exercise: Simple Tasks

- Write a program using tasks that will “randomly” generate one of two strings:
  - I think race cars are fun
  - I think car races are fun
- Hint: use tasks to print the indeterminate part of the output (i.e. the “race” or “car” parts).
- This is called a “Race Condition”. It occurs when the result of a program depends on how the OS schedules the threads.
- NOTE: A “data race” is when threads “race to update a shared variable”. They produce race conditions. Programs containing data races are undefined (in OpenMP but also ANSI standards C++’11 and beyond).

```
#pragma omp parallel
#pragma omp task
#pragma omp single
```

Tasks Example: Racey Cars

```
#include <stdio.h>
#include <omp.h>
int main()
{  printf("I think");
   #pragma omp parallel
   {
      #pragma omp single
      {
         #pragma omp task
            printf(" car");
         #pragma omp task
            printf(" race");
      }
   }
   printf("s");
   printf(" are fun!\n");
}
```

The output can sometimes be:
I think car races are fun!

and sometimes be:
I think race cars are fun!

When are tasks guaranteed to complete

- Tasks are guaranteed to be complete at thread barriers:

  #pragma omp barrier

- or task barriers

  #pragma omp taskwait

```
#pragma omp parallel
{
   #pragma omp task
   foo();
   #pragma omp barrier
   #pragma omp single
   {
      #pragma omp task
      bar();
   }
}
```

Multiple foo tasks created here – one for each thread

All foo tasks guaranteed to be completed here

One bar task created here

bar task guaranteed to be completed here

Example

```
#pragma omp parallel
{
  #pragma omp single
   {
     #pragma omp task
        fred();
     #pragma omp task
        daisy();
     #pragma taskwait
     #pragma omp task
        billy();
   }
}
```

`fred()` and `daisy()` must complete before `billy()` starts

Parallel Linked List Traversal

Serial code:

```
p = listhead ;
while (p) {
  process(p);
  p=next(p) ;
}
```

- Classic linked list traversal
- Do some work on each item in the list
- Assume that items can be processed independently
- Cannot use an OpenMP worksharing-loop directive

```
#pragma omp parallel
{
  #pragma omp single
   {
    p = listhead ;
    while (p) {
       #pragma omp task firstprivate(p)
              {
                process (p);
              }
       p=next (p) ;
     }
   }
}
```

Only one thread packages tasks

Makes a copy of `p` when the task is packaged

OpenMP Tasks: Data Scoping Defaults

- The behavior you want for tasks is usually **firstprivate**, because the task may not be executed until later (and variables may have gone out of scope)
  - **Variables that are private when the task construct is encountered are firstprivate by default**
- Variables that are shared in all constructs starting from the innermost enclosing parallel construct are shared by default

```
#pragma omp parallel shared(A) private(B)
{
    ...
#pragma omp task
    {
        int C;
        compute(A, B, C);
    }
}
```

A is shared
B is firstprivate
C is private

Example: Fibonacci numbers

```
int fib (int n)
{
   int x,y;
   if (n < 2) return n;

   x = fib(n-1);
   y = fib (n-2);
   return (x+y);
}

Int main()
{
   int NW = 5000;
   fib(NW);
}
```

- $F_n = F_{n-1} + F_{n-2}$
- Inefficient $O(n^2)$ recursive implementation!

Data Scoping with tasks: Fibonacci example.

This is an instance of the divide and conquer design pattern

```
int fib ( int n )
{

int x,y;
   if ( n < 2 ) return n;
#pragma omp task
   x = fib(n-1);
#pragma omp task
   y = fib(n-2);
#pragma omp taskwait
   return x+y
}
```

n is private in both tasks

x is a private variable
y is a private variable

What's wrong here?

A task's private variables are undefined outside the task

Data Scoping with tasks: Fibonacci example.

```
int fib ( int n )
{

int x,y;
   if ( n < 2 ) return n;
#pragma omp task shared (x)
   x = fib(n-1);
#pragma omp task shared(y)
   y = fib(n-2);
#pragma omp taskwait
   return x+y;
}
```

n is private in both tasks

x & y are shared
Good solution
we need both values to compute the sum

Parallel Fibonacci

```
int fib (int n)
{   int x,y;
   if (n < 2) return n;

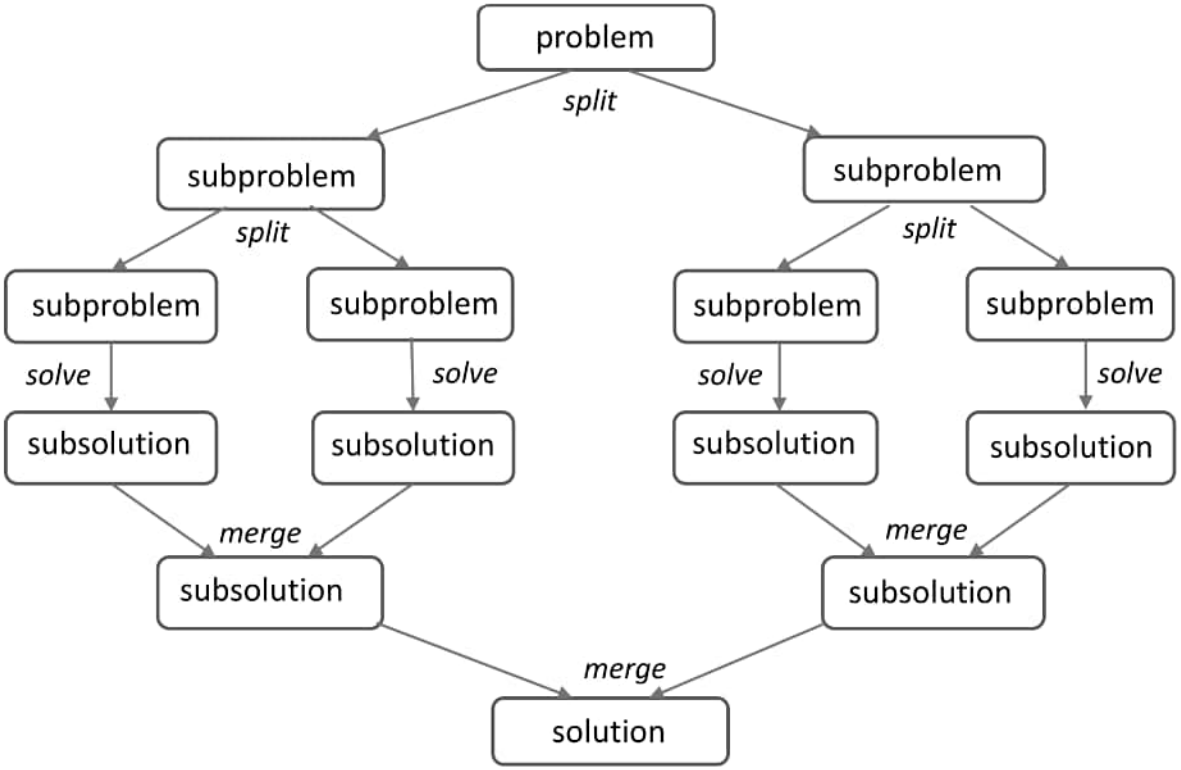
#pragma omp task shared(x)
   x = fib(n-1);
#pragma omp task shared(y)
   y = fib (n-2);
#pragma omp taskwait
   return (x+y);
}

Int main()
{  int NW = 5000;
   #pragma omp parallel
   {
      #pragma omp single
            fib(NW);
   }
}
```

- Binary tree of tasks
- Traversed using a recursive function
- A task cannot complete until all tasks below it in the tree are complete (enforced with taskwait)
- `x,y` are local, and so by default they are private to current task
  - must be shared on child tasks so they don't create their own firstprivate copies at this level!

Divide and Conquer

- Split the problem into smaller sub-problems; continue until the sub-problems can be solve directly



3 Options:

- Do work as you split into sub-problems
- Do work only at the leaves
- Do work as you recombine

Exercise: Pi with Tasks

- Consider the program Pi_recur.c. This program uses a recursive algorithm in integrate the function in the pi program.
  - Parallelize this program using OpenMP tasks

```
#pragma omp parallel
#pragma omp task
#pragma omp taskwait
#pragma omp single
double omp_get_wtime()
int omp_get_thread_num();
int omp_get_num_threads();
```

Pi with Tasks

```
#include <omp.h>
static long num_steps = 100000000;
#define MIN_BLK  10000000
double pi_comp(int Nstart,int Nfinish,double step)
{   int i,iblk;
   double x, sum = 0.0,sum1, sum2;
   if (Nfinish-Nstart < MIN_BLK){
      for (i=Nstart;i< Nfinish; i++){
         x = (i+0.5)*step;
         sum = sum + 4.0/(1.0+x*x);
      }
   }
   else{
      iblk = Nfinish-Nstart;
      #pragma omp task shared(sum1)
           sum1 = pi_comp(Nstart, Nfinish-iblk/2,step);
      #pragma omp task shared(sum2)
           sum2 = pi_comp(Nfinish-iblk/2, Nfinish,  step);
      #pragma omp taskwait
         sum = sum1 + sum2;
   }return sum;
}
```

```
int main ()
{
  int i;
  double step, pi, sum;
   step = 1.0/(double) num_steps;
   #pragma omp parallel
   {
      #pragma omp single
          sum =
              pi_comp(0,num_steps,step);
    }
     pi = step * sum;
}
```

Recursive divide-and-conquer algorithm

Pi OpenMP Results

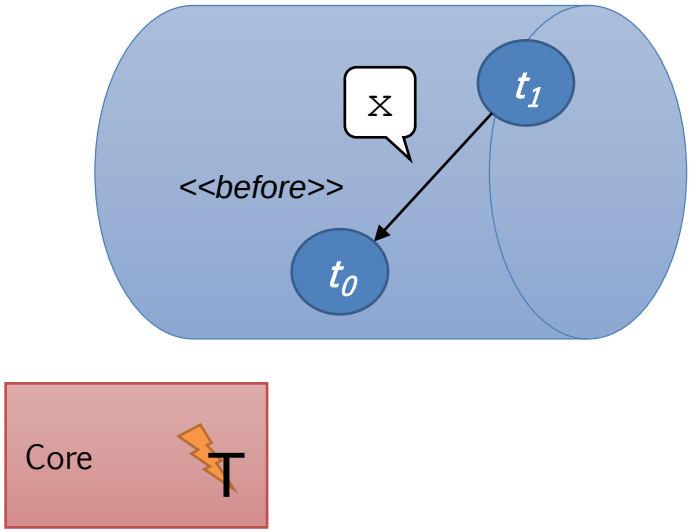
threads	1st SPMD	1st SPMD padded	SPMD critical	PI Loop and reduction	Pi tasks
1	1.86	1.86	1.87	1.91	1.67
2	1.03	1.01	1.00	1.02	1.00
3	1.08	0.69	0.68	0.80	0.76
4	0.97	0.53	0.53	0.68	0.52

- 1st SPMD uses array for sum ,has false sharing, hurt speedup.
- 1st SPMD uses padded array for sum, avoided false sharing, good speedup.
- SPMD critical uses scalar for sum, no false sharing, good speedup.
- Pi Loop and reduction uses scalar for partial sum, then reduction, relative good speedup, with some loop overhead.
- Pi tasks performs surprisingly well in this test. It is not reproducible on the HPC systems we tested with various compilers. Normally, do no use tasks if an algorithm is already well supported by OpenMP, such as standard do/for loop.

*Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

The depend clause

› Set a variable to act as placeholder for the dependency




```
#pragma omp parallel
{
  #pragma omp single
  {
    // Dependency is represented as a var
    int x = 0;

    #pragma omp task depend(in:x)
    {
      t0();
    }

    #pragma omp task depend(out:x)
    {
      t1();
    }
  } // bar&TSO
} // parreg end
```

Using Tasks

- Don’t use tasks for things already well supported by OpenMP
- For example standard do/for loops
- The overhead of using tasks is greater
- Don’t expect miracles from the runtime
  - Best results usually obtained where the user controls the number and granularity of tasks

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- **OpenMP and Performance**
- Recap, Q&A

The Truth

OpenMP and Performance

Can Get Good Performance And Scalability

If You Do Things Right

Easy $\neq$ Stupid

The Basics for All Users

OpenMP

Do Not Parallelize What Does Not Matter
(never tune a code without profile)

Do Not Share Data Unless You Have To
(exploit private data as much as you can)

One "Parallel For" Is Fine,
Multiple Back to Back Is Evil
(merge them where you can)

Think BIG
(maximize the size of the parallel regions)

The wrong and right way

```
#pragma omp parallel for
{ code block #1}
        ⋮
#pragma omp parallel for
{ code block #n}
```

Overhead of parallel region is repeated <n> times
No potential for “nowait” clause

```
#pragma omp parallel
{
  #pragma omp for
  { code block #1}
        ⋮
  #pragma omp for nowait
  { code block #n}
} // End of par region
```

Overhead of parallel region only once
Potential for “nowait” clause

Parallel in inner loops

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        #pragma omp parallel for
        for (k=0; k<n; k++)
            { .........}
```

Bad: n^2 overheads of parallel

```
#pragma omp parallel
   for (i=0; i<n; i++)
       for (j=0; j<n; j++)
           #pragma omp for
           for (k=0; k<n; k++)
               { .........}
```

Better: 1 overhead of parallel

The Basics (Yes, Also for You)

Every Barrier Matters
(but no more than necessary, please)

Do Not Lock Yourself Out
(use atomic constructs where possible)

Everything Matters
(minor bottlenecks add up too)

Why?
Amdahl's Law!

When Do Things Get Harder

OpenMP

Memory access "just happens"

There are however 2 cases to watch out for

NUMA and False Sharing

They have nothing to do with OpenMP though and are a charateristic of using a shared memory system

NUMA

OpenMP

With NUMA, the data access time varies

The time depends on where the data is

There are techniques to avoid this

This is covered in the OpenMP books

False Sharing

OpenMP

With false sharing, multiple threads access the same cache line in rapid succession

This quickly ruins parallel performance

There are techniques to avoid this

Throughout this tutorial this is covered

Outline

- Introduction
- Why Common Core?
- OpenMP Basics
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- OpenMP and Performance
- **Recap, Q&A**

The OpenMP Common Core: Most OpenMP Programs Only Use These 21 items

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads.
void omp_set_thread_num() int omp_get_thread_num() int omp_get_num_threads()	Default number of threads and internal control variables. SPMD pattern: Create threads with a parallel region and split up the work using the number of threads and the thread ID.
double omp_get_wtime()	Speedup and Amdahl's law. False sharing and other performance issues.
setenv OMP_NUM_THREADS N	Setting the internal control variable for the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies.
reduction(op:list)	Reductions of values across a team of threads.
schedule (static [,chunk]) schedule(dynamic [,chunk])	Loop schedules, loop overheads, and load balance.
shared(list), private(list), firstprivate(list)	Data environment.
default(none)	Force explicit definition of each variable's storage attribute
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive).
#pragma omp single	Workshare with a single thread.
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.

Beyond OpenMP Common Core

- **Synchronization mechanisms**
  - locks, flush and several forms of atomic
- **Data environment**
  - lastprivate, threadprivate, default(private|shared)
- **Fine grained task control**
  - dependencies, tied vs. untied tasks, task groups, task loops …
- **Vectorization constructs**
  - simd, uniform, simdlen, inbranch vs. nobranch, ….
- **Map work to do onto an attached device**
  - target, teams distribute parallel for, target data …
- … and much more. OpenMP 5.0 specification is over 600 pages!!!

Don't become overwhelmed. Master the common core and move on to other constructs when you encounter problems that require them.

Shared and private data

You have already seen some of the basics:

- Data declared outside a parallel region is shared.
- Data declared in the parallel region is private.
  (Fortran does not have this block scope mechanism)

```
int i;
#pragma omp parallel
{ double i; .... }
```

- You can change all this with clauses:

```
int i;
#pragma omp parallel private(i)
```

Variables in loops

```
int i; double t;
#pragma omp parallel for
  for (i=0; i<N; i++) {
    t = sin(i*pi*h);
    x[i] = t*t;
  }
```

- The loop variable is automatically private.
- The temporary `t` is shared, but conceptually private to each iteration: needs to be declared private.
  (What happens if you don't?)

Copying to/from private data

- Private data is uninitialized

```
int i = 3;
#pragma omp parallel private(i)
  printf("%d\n",i); // undefined!
```

- To import a value:

```
int i = 3;
#pragma omp parallel firstprivate(i)
  printf("%d\n",i); // undefined!
```

- `lastprivate` to preserve value of last iteration.

Default behaviour

- `default(shared)` or `default(private)`
- useful for debugging: `default(none)`
  because you have to specify everything as shared/private

Persistent thread data

- Private data disappears after the parallel region.
  What if you want data to persist?
- Directive `threadprivate`

```
double seed;
#pragma omp threadprivate(seed)
```

- Standard application: random number generation.
- Tricky: has to be global or static.

Arrays

- Statically allocated arrays can be made private.
- Dynamically allocated ones can not: the pointer becomes private.

Need for synchronization

- The loop and sections directives do not specify an ordering, sometimes you want to force an ordering.
- Barriers: global synchronization.
- Critical sections: only one process can execute a statement this prevents race conditions.
- Locks: protect data items from being accessed.

Barriers

- Every workshare construct has an implicit barrier:

```
#pragma omp parallel
{
  #pragma omp for
    for ( .. i .. )
      x[i] = ...
  #pragma omp for
    for ( .. i .. )
      y[i] = .. x[i] .. x[i+1] .. x[i-1] ...
}
```

  First loop is completely finished before second.

- Explicit barrier:

```
#pragma omp parallel
{
  x = f();
#pragma omp barrier
  .... x ...
}
```

Critical sections

- Critical section: One update at a time.

```
#pragma omp parallel
{
  double x = f();
#pragma omp critical
  global_update(x);
}
```

- `atomic` : special case for simple operations, possible hardware support

```
#pragma omp atomic
  t += x;
```

Warning

- Critical sections are not cheap! The operating system takes thousands of cycles to coordinate the threads.
- Use only if minor amount of work.
- Do not use if a reduction suffices.
- Name your critical sections.
- Explore locks if there may not be a data conflict.

Locks

- Critical sections are coarse:
  they dictate exclusive acces to a *statement*
- Suppose you update a big table
  updates to non-conflicting locations should be allowed
- Locks protect a single data item.

Locks

```
omp_lock_t writelock;

omp_init_lock(&writelock);

#pragma omp parallel for
for ( i = 0; i < x; i++ )
{
    // some stuff
   omp_set_lock(&writelock);
    // one thread at a time stuff
    omp_unset_lock(&writelock);
    // some stuff
}

omp_destroy_lock(&writelock);
```

```
#include <stdio.h>
#include <omp.h>

int main()
{
        int var = 0;
        // init lock
        omp_lock_t lock;

        omp_init_lock(&lock);

        #pragma omp parallel num_threads(1024) shared(lock)
        {
                // set lock
                omp_set_lock(&lock);

                // increment var
                var++;

                // unset lock
                //omp_unset_lock(&lock);

        } // barrier

        printf("var is %d (should be 1024)\n", var);

        // destroy lock
        omp_destroy_lock(&lock);


        return 0;
}
```

Producer and consumer with openmp

```
#pragma omp parallel default(none) shared(buffer) private(tid)
{
      tid = omp_get_thread_num();
      if (tid == 0) consumer(buffer);
      else producer(buffer);
   }
}

int get_value(buffer_type &buffer)  {
   while (true) {
      omp_set_lock(&buffer.lock);
      if ( buffer.n > 0 ) {
         buffer.n--;
         int ret = buffer.value[buffer.n];
         omp_unset_lock(&buffer.lock);
         return ret;
      }
      omp_unset_lock(&buffer.lock);
   }
}


void consumer(buffer_type &buffer)
{
   while (true) {
      int value = get_value(buffer);
      solve_problem(value);
   }
}
```

```
void put_value(buffer_type &buffer, int val)
{
   while (true) {
      omp_set_lock(&buffer.lock);
      if ( buffer.n < buffer.size ) {
         buffer.value[buffer.n] = val;
         buffer.n++;
         omp_unset_lock(&buffer.lock);
         return;
      }
      omp_unset_lock(&buffer.lock);
   }
}

void producer(buffer_type &buffer)
{
   while (true) {
      // Compute a value
      int value = compute_problem();
      // Insert in buffer
      put_value(buffer, value);
   }
}
```

Advanced OpenMP Topics

Joachim Protze

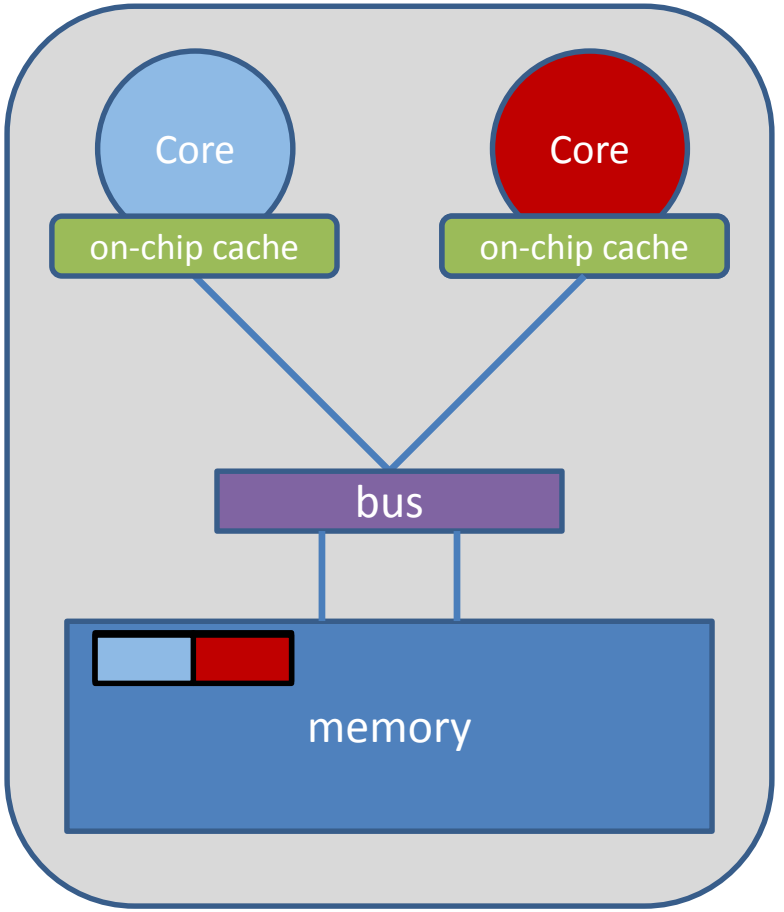
IT Center, RWTH Aachen University

protze@itc.rwth-aachen.de

Many slides provided by: Christian Terboven

Recall: False-Sharing

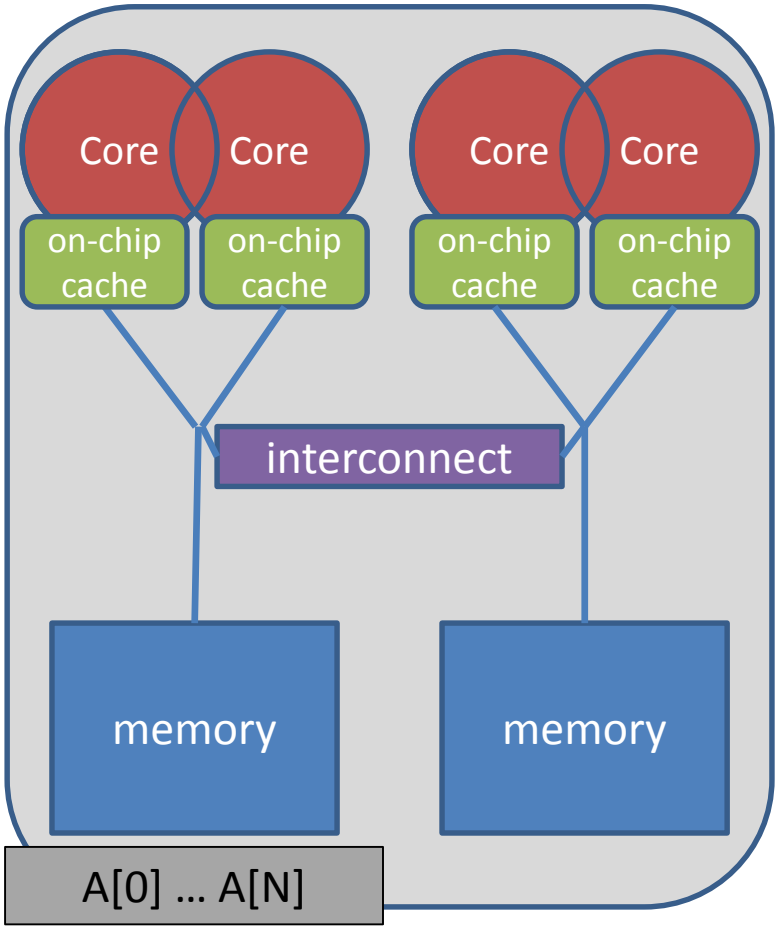
- **False Sharing occurs when**
  - → different threads use elements of the same cache-line
  - → one of the threads writes to the cache-line
- **As a result the cache line is moved between the threads, although there is no real dependency**
- **Note: False Sharing is a performance problem, not a correctness issue**



How To Distribute The Data ?

```
double* A;
A = (double*)
    malloc(N * sizeof(double));
```

```
for (int i = 0; i < N; i++) {
   A[i] = 0.0;
}
```

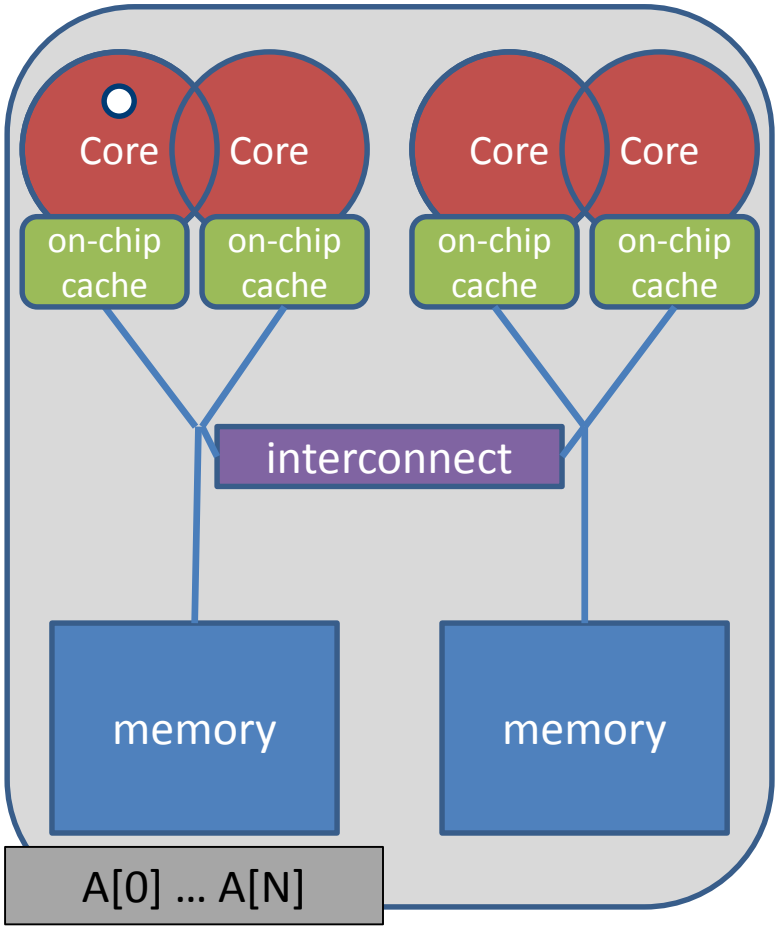


Non-Uniform Memory Arch.

- **Serial code: all array elements are allocated in the memory of the NUMA node containing the core executing this thread**

```
double* A;
A = (double*)
    malloc(N * sizeof(double));
```

```
for (int i = 0; i < N; i++) {
   A[i] = 0.0;
}
```



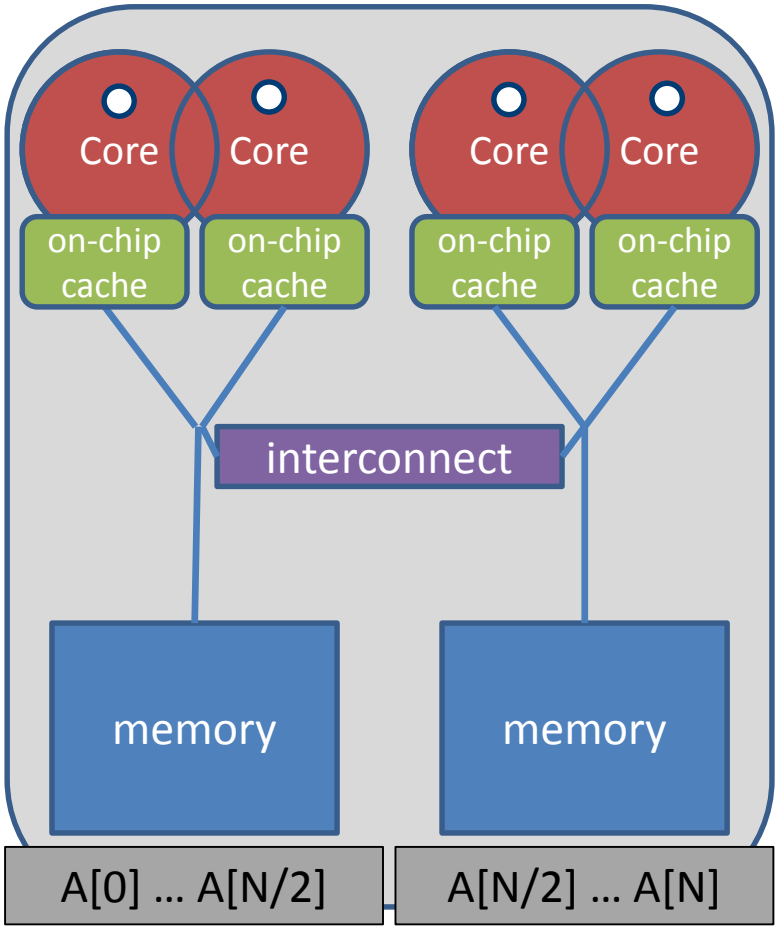
Non-Uniform Memory Arch.

- First Touch w/ parallel code: all array elements are allocated in the memory of the NUMA node containing the core executing the thread initializing the respective partition

```
double* A;
A = (double*)
     malloc(N * sizeof(double));

omp_set_num_threads(4);

#pragma omp parallel for
for (int i = 0; i < N; i++) {
   A[i] = 0.0;
}
```



Decide for Binding Strategy

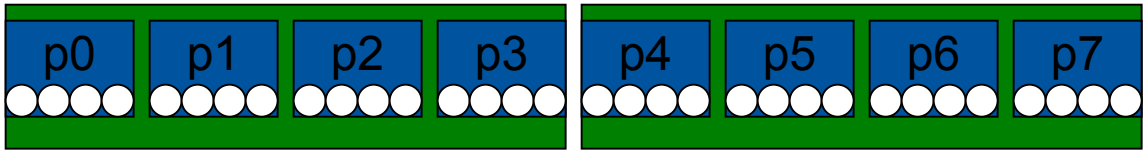
- **Selecting the „right“ binding strategy depends not only on the topology, but also on the characteristics of your application.**
  - → Putting threads far apart, i.e. on different sockets
    - →May improve the aggregated memory bandwidth available to your application
    - →May improve the combined cache size available to your application
    - →May decrease performance of synchronization constructs
  - → Putting threads close together, i.e. on two adjacent cores which possibly shared some caches
    - →May improve performance of synchronization constructs
    - →May decrease the available memory bandwidth and cache size
- **If you are unsure, just try a few options and then select the best one.**

OpenMP 4.0: Places + Binding Policies (1/2)

- **Define OpenMP Places**
  - → set of OpenMP threads running on one or more processors
  - → can be defined by the user, i.e. `OMP_PLACES=cores`
- **Define a set of OpenMP Thread Affinity Policies**
  - → SPREAD: spread OpenMP threads evenly among the places
  - → CLOSE: pack OpenMP threads near master thread
  - → MASTER: collocate OpenMP thread with master thread
- **Goals**
  - → user has a way to specify where to execute OpenMP threads for
  - → locality between OpenMP threads / less false sharing / memory bandwidth

Places

- **Assume the following machine:**



→ 2 sockets, 4 cores per socket, 4 hyper-threads per core

- **Abstract names for OMP_PLACES:**
  - → threads: Each place corresponds to a single hardware thread on the target machine.
  - → cores: Each place corresponds to a single core (having one or more hardware threads) on the target machine.
  - → sockets: Each place corresponds to a single socket (consisting of one or more cores) on the target machine.

OpenMP 4.0: Places + Binding Policies (2/2)

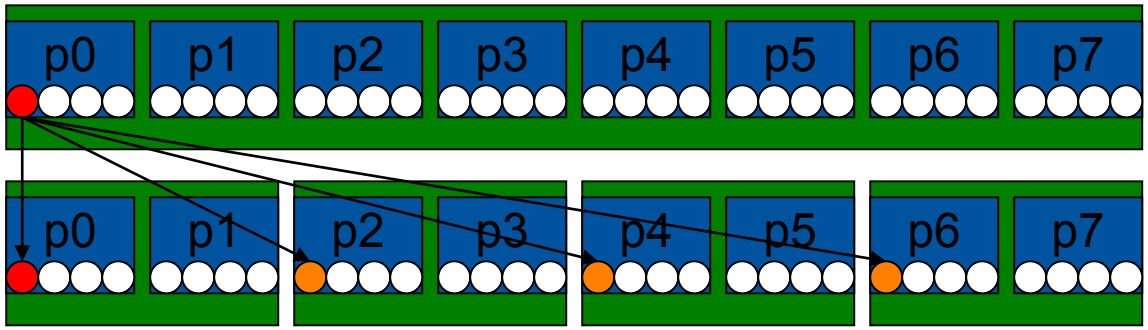
- **Example**

```
OMP_PLACES = cores
OMP_NUM_THRADS = 4

#pragma omp parallel proc_bind(spread)
```

→ initial

→ spread



OpenMP 4.0: Places + Binding Policies (2/2)

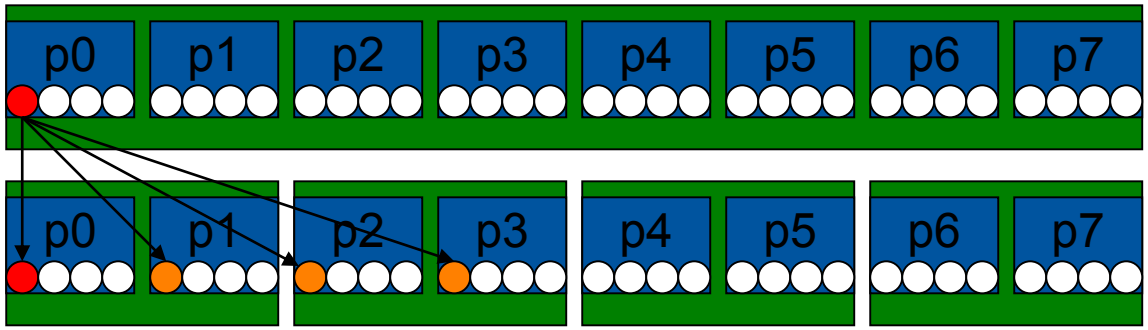
- **Example**

```
OMP_PLACES = cores
OMP_NUM_THRADS = 4

#pragma omp parallel proc_bind(close)
```

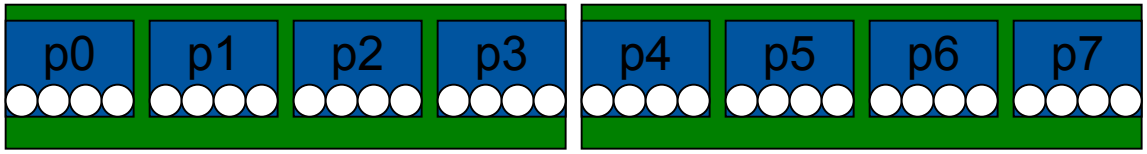
→ initial

→ close



Upcoming: Binding Teams

- **Assign different teams to each socket**



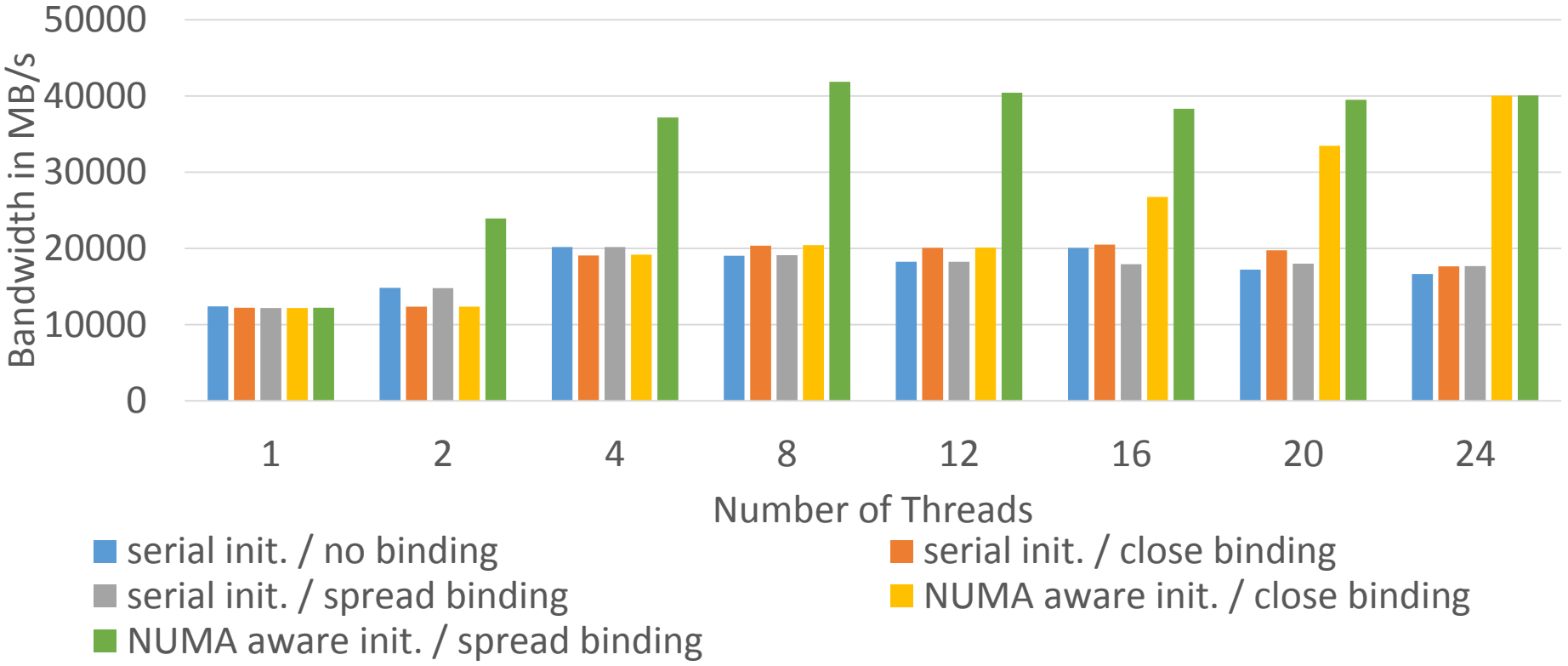

```
OMP_PLACES = cores

#pragma omp teams num_teams(NUM_SOCKETS) proc_bind(spread)
#pragma omp parallel num_threads(4) proc_bind(spread)
```

- **OpenMP synchronization is limited to a team**
  - → Reduce the latency of synchronization
  - → Allow to explicitly partition a problem into independent domains
- **Current alternative: If you use hybrid MPI+OpenMP,**

Serial vs. Parallel Initialization

- **Performance of OpenMP-parallel STREAM vector assignment measured on 2-socket Intel® Xeon® X5675 („Westmere“) using Intel® Composer XE 2013 compiler with different thread binding options:**



If I'm writing new code, how do I choose between Intel Cilk Plus, TBB, OpenMP, and MPI?

It's not an exclusive choice. However let us focus on "how to multi-thread" question. First, you'll want to look at the **development environment**. If the code is written in FORTRAN, use OpenMP. TBB works only for C++, and makes heavy use of templates. If the code is C, or you are not comfortable with templates, your choice is narrowed to Cilk Plus or OpenMP. If you need parallelism with non-Cilk Plus enabled compilers, your choice is narrowed to OpenMP or TBB. If you need parallelism in C++ with compilers that do not support Cilk Plus or OpenMP, then TBB is the way to go, because it does not require compiler support.

If the code makes heavy use of **recursion**, then Cilk Plus or TBB is likely a better fit than OpenMP. Cilk Plus has particularly low cost "spawning", so it's a particularly good fit for highly recursive code. If the code is C++, it's likely that TBB is the best fit. TBB matches especially well with the code that is highly object oriented, and makes heavy use of C++ templates and user defined types. If the code is written in C or FORTRAN, OpenMP may be the better solution because it fits better than TBB into a structured coding style and for simple cases, it introduces less coding overhead.

If I'm writing new code, how do I choose between Intel Cilk Plus, TBB, OpenMP, and MPI?

How much are you willing to make your code stray from plain serial code? If you use Cilk Plus without array notation, then the code still **looks like serial code** with some markup. In contrast, conversion to TBB will require restructuring certain parts to fit the TBB templates.

Next, look at **what you want to make parallel**. Use OpenMP if the parallelism is primarily for bounded loops over built-in types, or if it is flat do-loop centric parallelism. OpenMP works especially well with large and predictable data parallel problems. It can be very challenging to match OpenMP performance with Cilk Plus or TBB for such problems. Cilk Plus and TBB excels at less structured or consistent parallelism.

Consider using TBB if you need off-the-shelf **patterns** that go beyond loop-based and recursion-based parallelism, since TBB provides generic parallel patterns for parallel while-loops, data-flow pipeline models, parallel sort, and prefix-scan.